LEARN · VERIFY — OFFLINE STUDY BOOKLET

Incident Response & Postmortems

An outage is a coordination problem before it is a technical one. Here is the structure that contains it, the habit that shortens it, the practice that makes both possible — and the write-up that stops it happening twice.

Print double-sided, short-edge bind, A4 · 100% scale (no “fit to page”).

Answer key starts at Section 9 — keep a sheet of paper over it.

How to use this booklet

The loop, per section

1. Read the plain-version box first. If it doesn't land, the rest won't either.
2. Read the teaching. Say each boundary out loud: “X is *this*, its look-alike Y is *that*.”
3. Cover the page. Write the answer to **Q1** in the blank lines, in your own words, in full sentences. **Then** check the key.
4. Score yourself on completeness. Then do **Q2** and note whether it beat Q1.
5. Only then move on.

> **THE RULE THAT DOES MOST OF THE WORK**
>
> If you find yourself thinking “yes, I know this” — that is *recognition*, and it is not learning. The only evidence that counts is a written answer produced with the page covered.

> **WHAT THIS SUBJECT ASKS OF YOU THAT OTHERS DON'T**
>
> Everything here has to work **while you are frightened**. That is the constraint the whole subject is built around, and it is why so much of the advice is about deciding things in advance. You cannot choose a chat channel, invent a role structure, or remember to be fair to a colleague while a service is down and your phone is going. So as you read, keep asking the question this competency is really made of: *which of these decisions am I making now, calmly, so that nobody has to make it at three in the morning?* The answer is almost always “more of them than I thought”.

The spacing schedule

Review at **day 1, day 3, day 7, day 16, day 35**. On a pass, jump to the next interval. On a fail, reset that item to 1 day. Use the grid at the back; one row per item.

What's in here

SECTION 1

Learning goal & roadmap

LEARNING GOAL — WHAT YOU SHOULD BE ABLE TO DO WHEN YOU CLOSE THIS

Handed a live incident, or a postmortem someone else wrote, you can:

1. **Impose a structure in under a minute** — declare, take command, name the two roles that report to you, and put everyone in one place.
2. **Order the work correctly**: assess, mitigate, then diagnose — and say why a mitigation that does not depend on understanding the cause is the one worth building in advance.
3. **Name what should have been decided before the pager went off**, and decide it.
4. **Read a postmortem and say whether it will change anything** — judging it on blame, specificity, ownership, priority, audience and speed, not on how thorough it feels.

The core model in one paragraph

An outage has two problems inside it, and they are not the same problem. There is the technical fault, and there is the **coordination** of everyone converging on it — and the second is the one that turns a bad hour into a bad day. Structure is the answer to the second: a single line of command, roles agreed beforehand, one place where everyone talks, and a written record kept as you go. That structure is worth nothing unless someone actually declares an incident, which is why "declare early and often" is a principle rather than a preference. Once the structure exists, the work has an order that people reliably get wrong: **stop the bleeding before you understand the wound**, because customers do not care whether you know what caused their errors, only that the errors have stopped. None of this can be improvised at three in the morning, so the real work happens beforehand — training, a chosen channel, a contact list, criteria for what counts as an incident, and drills that expose the gaps while the stakes are low. And when it is over, the only thing that carries forward is the write-up: blameless enough that people tell the truth, specific enough to act on, owned and prioritised so the actions actually close, and published widely enough that someone who was not there learns something.

TENTH-GRADER VERSION — THE WHOLE THING AT ONCE

- When something big breaks, the hard part usually isn't the bug. It's ten people all helping at once and none of them knowing who is deciding.
- So: one person is in charge. One person fixes. One person talks to everyone else. Everybody in the same room, real or virtual.
- Say "this is an incident" out loud, early. It is much cheaper to stand a response down than to start one late.
- Stop the damage first. You do not need to know *why* to make it stop — you only need to know roughly *where*.
- Everything that has to work when you're panicking has to be decided when you're calm. Which chat channel. Who to call. What even counts as an incident.
- Afterwards, write it down without blaming anyone, with real numbers, with named owners and priorities — and then actually do the things. A write-up nobody acts on is the same as no write-up.

The four parts, and why they're in this order

PART	WHAT IT COVERS	WHY HERE

1 The structure, before you need it	Managing an incident versus resolving it; the four basic principles; the framework the whole practice is borrowed from; the three goals it exists to serve; and the three roles — commander, operations, communications — with how they nest, delegate and collapse back.	Every later part assumes someone is in charge. This is the part that puts them there, and it is the part most often skipped because the incident “doesn't feel big enough yet”.
2 Mitigate first, understand later	The four-step order of an active incident; generic mitigations versus bespoke ones; why knowing the *location* of a cause is enough to act; when to build the tools that make it possible; and three worked incidents — one mishandled, one large and messy, one handled well.	Structure without the right ordering just coordinates people into the wrong work. This part is where minutes are actually saved, and where the biggest single habit lives.
3 Practise before the pager	What to teach responders; the logistics to settle in advance — channel, templates, contacts, criteria; and drills, from company-scale controlled emergencies down to treating a small problem as a big one on purpose.	Parts 1 and 2 describe behaviour under stress, and stressed people do what they have practised. This part is how the previous two become reflexes instead of good intentions.
4 The write-up that changes something	A worked case with two postmortems of the same outage, one bad and one good; the eight ways a write-up fails; the seven qualities of one that works; the incentives that keep the culture alive; and the four signs that it is quietly dying.	The last part because it is the only one that outlives the incident. An organisation that is good at parts 1–3 and bad at this one relives the same outage indefinitely.

> *“By the time a pager alerts you to a problem, it's too late to think about how to manage the incident.”*

1

PART 1 OF 4

The structure, before you need it

What an incident response framework is for, and the three roles that do the work.

TENTH-GRADER VERSION

- Two different jobs are happening at once: fixing the thing, and running the people who are fixing the thing. They are not the same job, and one person doing both is how it goes wrong.
- The whole approach is borrowed from firefighting. It has been around since 1968 and was built for exactly this: strangers turning up to one emergency and needing to work together immediately.
- Three goals: coordinate, communicate, stay in control. When incident response goes wrong, it is nearly always one of those three.
- Three roles: the commander decides, the operations lead fixes, the communications lead talks. The commander holds every role they haven't handed out yet.
- Everyone in one place — one room, or one channel. Not five side conversations.
- Write things down as you go. Three or more people means someone should be keeping a document of what you've tried and ruled out.

THE EVERYDAY VERSION

A child goes missing on a school trip. Within a minute there are fifteen adults, all of whom want to help, and that is the danger — not the shortage of effort but the shape of it. What actually works is somebody saying, out loud, three things: *I am coordinating this*; *you two search the east side and report back to me*; *you phone the parents and the school and keep phoning them*. Everything else follows from those sentences. Note what they do. They establish one person whose job is to decide rather than to search. They split the doing from the telling, because the person crawling under the bleachers cannot also be reassuring a parent. And they stop the other twelve adults from starting twelve uncoordinated searches, each of which will later have to be repeated because nobody knows where anyone has already looked. That is the entire content of this part, and none of it is about children.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Resolving an incident	Mitigating the impact **and/or** restoring the service to its previous condition.	This is the technical work: the thing everyone instinctively runs toward.	vs **managing** an incident, below. The two words get used interchangeably and they name different jobs. Resolving is about the system; managing is about the people. A response can resolve the fault and still be badly managed, and that is the common case.

Managing an incident	Coordinating the efforts of responding teams in an efficient manner, and ensuring that communication flows **both** between the responders **and** to those interested in the incident's progress.	When a not-so-ordinary, urgent problem requires multiple individuals or teams, you are suddenly faced with doing both at once — managing the response and resolving the problem.	Note the two directions of communication. Inward, between responders, is what stops duplicated and contradictory work. Outward, to everyone watching, is what stops the second incident: the one where the organisation concludes nothing is being done.
The four basic principles	The premise is to respond in a **structured way**, because large-scale incidents can be confusing and a structure teams agree on beforehand reduces chaos.	**Maintain a clear line of command. Designate clearly defined roles. Keep a working record of debugging and mitigation as you go. Declare incidents early and often.**	Formulating the rules for communication and coordination *before* disaster strikes is what allows the team to concentrate on resolving the incident when it happens. A team that has already practised does not have to spend attention on these factors at all — which is the actual benefit, and it is a benefit to *attention*, not to procedure.
Incident Command System (ICS)	The framework the whole practice derives from. Established in **1968 by firefighters** as a way to manage wildfires; it provides standardized ways to communicate and fill clearly specified roles during an incident.	Based on the model's success, companies later adapted it to respond to computer and system failures. Two well-known adaptations exist: one from a large technology company, and one from a company whose own product is incident management.	vs a process you must adopt whole. The full approach may be more than you need; you can build a framework by **selecting the parts that matter to your organisation**. It is a simple concept, easily understood, and it scales up or down with the size of the company and of the incident — but simple to understand is not the same as easy to implement.
The three Cs	The common goals of every incident response framework: **Coordinate** the response effort; **Communicate** between responders, within the organisation, and to the outside world; and maintain **Control** over the incident response.	When something goes wrong with incident response, **the culprit is likely in one of these three areas.**	Note what is not on the list: fixing the fault. The 3Cs are goals for the *response*, not for the system. Mastering them is essential to responding effectively — which is a claim about the response, not about your uptime.
Incident Commander (IC)	Leads the incident response and directs its high-level state. **The person who declares the incident typically steps into this role.**	The IC commands and coordinates the response, delegating roles as needed; communicates effectively; stays in control of the response; and works with other responders to resolve the incident. The roles are organised as a hierarchy — the other two report to the IC.	The default that makes the system work with one person: **the IC assumes all roles that have not been delegated yet.** So there is never a gap. The IC may either hand the command role to someone else and take operations themselves, or assign operations to someone else — both are normal, and one of the worked cases turns on a commander handing over to a more experienced colleague mid-incident.
Operations Lead (OL)	Works to respond to the incident by **applying operational tools** to mitigate or resolve it.	In one worked case the commander delegated the ordinary problems — restoring power, rebooting servers — to the operations lead, and engineers reported their progress back through that lead.	vs the commander. The distinction is that the OL acts on the system and the IC decides what should be acted on. The OL may lead a team, and those teams expand or contract as needed.

Communications Lead (CL)	The **public face** of the incident response team. Provides periodic updates to the response team and to stakeholders, and manages inquiries about the incident.	In the power-outage case the CL observed and asked questions about all incident-related activities, and reported accurate information to four distinct audiences: company leaders, teams with storage concerns, external customers, and specific customers who had filed support tickets.	The role is elastic in one direction only: **if the incident becomes small enough, the CL role can be subsumed back into the IC role.** It is created by delegation and dissolved by absorption, which is why an incident does not need to be big before it can be managed.
Centralized communication	Gathering **all** the responders in one place and communicating in real time. The war room can be a physical location like a conference room, or virtual — a chat channel or a call.	It is named as an important principle of the response protocol, and it is the first thing the commander does in two of the worked cases.	vs communicating *about* the incident in the bug tracker. In the mishandled case, the team never declared an incident and kept troubleshooting through "normal" methods, using the bug tracking system for communication — and miscommunication between two groups of developers led them down the wrong path during troubleshooting.
The working document	When **three or more people** work on an incident, start a collaborative document listing working theories, eliminated causes, and useful debugging information such as error logs and suspect graphs.	Its purpose is stated precisely: the document **preserves this information so it doesn't get lost in the conversation**.	It also gives you somewhere to put the detail when the main channel gets too busy. In the large case, discussion became too dense for the chat channel, so detailed debugging moved to the shared document and the channel became **the hub for decision making** — a split worth copying deliberately.

> **CONCEPT BOUNDARY — DECLARING AN INCIDENT IS A CHEAP, REVERSIBLE ACT**
>
> **What people think it is:** an escalation with a cost — waking people, alarming management, admitting the situation is out of hand. So they wait until they are sure.
> **What it actually is:** the act that gives you the tools. Declaring is what creates the command line, the roles, the shared channel and the record. Until it happens you are a group of individuals with a bug number.
> **What waiting costs, specifically:** declaring early prevents miscommunication between separate groups of developers; makes root-cause identification and resolution happen sooner; and loops relevant teams in earlier, which makes external communication faster and smoother. Those three are the stated benefits, and every one of them is a benefit you forfeit by waiting rather than a cost you defer.
> **The rule that follows:** declare early *and often*. Standing a response down is trivial; starting one two hours late is not.

Anchor sketch — who holds what, and how it collapses

Legend: bold = a named role, goal or principle **reversed = the act that creates all of it** **Q = cover the page and answer this**

```
two jobs are running at once, and they are different:
   RESOLVING = mitigate the impact and/or restore
               the service to its previous condition
   MANAGING  = coordinate the responding teams, and
               keep communication flowing BOTH ways
               (between responders, and outward)
                        |
SOMEONE DECLARES AN INCIDENT  <- everything below
exists only after this sentence is said out loud.
whoever declares it TYPICALLY becomes the commander.
                        |
THE THREE Cs  what the framework is FOR
     COORDINATE the response effort
     COMMUNICATE between responders, inside the org,
                 and to the outside world
     CONTROL     stay in control of the response
     +--> when incident response goes wrong, the culprit
          is LIKELY in one of these three.
                        |
THE THREE ROLES  a hierarchy: the other two report
up to the commander
     INCIDENT COMMANDER . decides, delegates, stays in
          control, works with responders to resolve
          +--> DEFAULT: the IC holds every role not yet
               delegated. so there is never a gap.
          +--> may hand OVER command and take ops, or
               assign ops to someone else. both normal.
     OPERATIONS LEAD .... applies operational tools to
          mitigate or resolve
     COMMUNICATIONS LEAD  the public face. periodic
          updates to the team and to stakeholders;
          handles inquiries
          +--> if the incident gets SMALL enough, this
               role is subsumed back into the IC.
                        |
FOUR BASIC PRINCIPLES
     clear line of command . clearly defined roles .
     a working record kept AS YOU GO .
     declare incidents EARLY AND OFTEN
                        |
everyone in ONE place (room, channel or call), talking
in real time. 3+ people -> start a shared document of
theories tried and causes eliminated, so it is not
lost in the conversation.
```

Q Give the 3Cs, and say what they are goals *for*.

Q Who is the commander by default, and which roles do they hold?

Q Name the three specific things declaring early is said to buy you.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN

Declaring an incident early	The command line, the roles, the shared channel and the record — plus earlier root-causing and smoother external communication.	The appearance of over-reacting, and other people's attention.	Always, on the stated rule: early *and often*. Standing down is cheap.
Handing over command mid-incident	The response gets whoever is best at running it, which may not be whoever noticed first.	A handover, at the worst possible moment, with context to transfer.	When someone with more incident-response experience is available — one worked case does exactly this, two hours in.
Splitting out a communications lead	The people fixing the thing stop being interrupted, and the outside world gets consistent, accurate updates.	A responder who is now not resolving anything.	As soon as anyone outside the response is asking. It collapses back into the commander when the incident shrinks.
One channel for everyone	Real-time coordination, and a record everyone can see.	It gets loud. In the large case a dozen participants made communication itself a challenge.	Always — and when it gets too dense, move detail to the shared document and keep the channel for decisions.
Keeping the record as you go	Theories and eliminated causes survive the conversation, and the postmortem is much easier to write.	Somebody's attention, during the incident, on writing rather than fixing.	Three or more responders. Below that the conversation is small enough to hold.
Adopting the framework whole	A complete, battle-tested structure with nothing left to invent.	More process than a small organisation needs, and more to teach.	Rarely at first — select the parts that matter to you and grow into the rest.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“Let's keep digging and see if it gets worse.”	“I'm declaring an incident and taking command. If it turns out to be nothing we stand down — that costs us far less than starting late.”
“Everyone's on it.”	“Who is the commander? Until that's answered, everyone being on it is the problem, not the reassurance.”
“I'll update the bug as I go.”	“Bug trackers are for tracking, not for coordinating. Everyone into the channel — real time, one place.”
“I'm running the response and also debugging it.”	“I hold every role I haven't delegated. Let me hand operations to you so I can stay on coordination.”
“Support keeps interrupting us for updates.”	“That's a comms lead's job. They're the public face; we stop context-switching and the outside world stops guessing.”
“This channel is unreadable now.”	“Move the debugging detail into the shared doc and keep the channel as the decision hub. Log the theories we've eliminated.”
“I noticed it, so I suppose I'm running it.”	“I am, by default — and I'd like to hand command to someone with more incident experience. I'll take operations.”

PART 1 · QUESTION 1

A payments API starts returning errors for a subset of merchants. The on-call engineer opens a ticket, starts investigating, and posts findings into the ticket as they go. Over the next ninety minutes four other engineers notice the ticket and begin investigating in parallel; two of them independently rule out the same subsystem. The support team, fielding merchant calls, is asking the on-call engineer for updates roughly every ten minutes. Nobody has used the word “incident”. The on-call engineer's view is that they will declare one “if it turns out to be serious”.

(a) Name the two distinct jobs now in progress, define each, and say which one has no owner. **(b)** Give the four basic principles, and identify which of them this situation violates and how. **(c)** Take the on-call engineer's reasoning seriously and say precisely what is wrong with it, using the three specific benefits that declaring early is said to provide. **(d)** Write the three sentences you would say to fix this in the next sixty seconds.

PART 1 · QUESTION 2

Three hours into a regional outage, the incident channel has nineteen participants. The commander — the engineer who first noticed the problem — is simultaneously answering questions from a director, running two debugging threads, and writing an update for the status page. A colleague who has run many incidents joins and offers to help. Separately, two responders have each spent forty minutes eliminating the same suspected cause without knowing about each other.

(a) Diagnose the commander's situation in terms of the default rule about undelegated roles, and say what specifically has gone wrong as a result. **(b)** Give both of the moves available to the commander now that an experienced colleague has arrived, and say what each costs. **(c)** Name the artefact that would have prevented the duplicated forty minutes, say at what team size it should have been started, and state exactly what it holds. **(d)** The channel is now too noisy to follow. Give the specific split to make, and say what stays in each place.

2×2 — has it been declared, and is anyone in command

COLUMNS: IS THERE A SINGLE, NAMED COMMANDER? →

	Yes — one person, and everyone knows who	No — several people helping
An incident HAS been declared	**The structure is doing its job** Command line, roles, one channel, a record kept as you go. The commander holds whatever has not been delegated, so nothing is unowned. *Move:* stay here. Delegate operations and communications as the incident grows, and let communications collapse back into command as it shrinks. Keep the shared document alive from the third responder onward.	**Declared, and drifting** The word was said but no one took command, so the roles never got assigned and the response is a crowd with a label. *Move:* somebody claims it, out loud, now — by default the person who declared it. An incident with no commander is worse than an undeclared one, because everyone believes it is being run.
Still "just a ticket"	**Quietly competent, and fragile** Someone senior is effectively running it without the word being used. It often works — until they go off shift, or the incident outgrows one person's head. *Move:* declare anyway. You lose nothing and you gain the record, the channel and the ability to hand over. The handover is the part that fails without it.	**Where the bad case starts** Parallel investigation in a bug tracker; support chasing individuals; duplicated work nobody can see; miscommunication between two groups sending them down the wrong path. *Move:* declare. That single act creates the command line, the roles, the shared channel and the record. Everything else in this booklet is downstream of it, which is why the principle is early *and often*.

Read it as: the row is a decision that takes one sentence, and the column is a decision that takes one more. Both are free, and both are routinely postponed until the situation is bad enough to "justify" them — by which point the cost has already been paid in duplicated work and silence.

CARRY-OUT RULE FROM THIS PART

If you are asking whether it is big enough to declare, declare it. The question itself is the signal: it only occurs to you once coordination has already become a problem. Declaring costs one sentence and is reversible in one more; not declaring costs duplicated investigations, an uninformed audience, and a handover you cannot perform.

2

PART 2 OF 4

Mitigate first, understand later

The order the work has to happen in, and the kind of fix that does not wait for a diagnosis.

TENTH-GRADER VERSION

- Four steps, in this order: work out who is hurt, stop the hurting, then find out why, then fix the why and write it up.
- Most people do it in the wrong order. They try to understand before they try to stop it, because understanding feels like the responsible thing.
- You do not need to know *what* broke to make it stop. You only need to know roughly *where*.
- A blunt fix that works without a diagnosis — roll it back, take that region out — is worth more in the first hour than a precise one you can't apply yet.
- Blunt fixes are blunt: they can cause their own problems. That is a price, not a reason to skip them.
- The tools that let you do this take time to build, so they have to be built before the emergency, not during it.
- Customers do not care whether you understand the outage. They care that the errors stopped.

THE EVERYDAY VERSION

Water is coming through the kitchen ceiling. There are two things you can do, and only one of them is urgent. You can go up to the bathroom with a torch and work out which joint has failed, how long it has been weeping, and whether the last plumber used the wrong fitting — or you can turn off the stopcock. The stopcock is a **generic mitigation**: a blunt instrument that works without any diagnosis at all, because it does not care *which* pipe is leaking. It has real costs — nobody in the house has water now — and it fixes nothing permanently. But it stops the damage in fifteen seconds, and it buys you the whole afternoon to find the joint calmly. The mistake, and it is the natural one, is to go up with the torch first, because that feels like the competent thing to do while the ceiling is still dripping. Note also that the stopcock only helps if you know where it is and it turns. That is a thing you find out on a dry day.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
The order of an active incident	With mitigation as top priority: **1.** Assess the impact. **2.** Mitigate the impact. **3.** Perform a root-cause analysis. **4.** After the incident is over, fix what caused it and write a postmortem.	Afterwards you can run response drills to exercise the vulnerabilities in the system, and engineers can work on projects to address them.	vs the order people actually use, which puts diagnosis second. The consequence is stated flatly: **first responders must prioritize mitigation above all else, or time to resolution suffers.**

Generic mitigation	An action first responders take to alleviate pain **even before the root cause is fully understood**.	Roll back a recent release when an outage is correlated with the release cycle; reconfigure load balancers to avoid a region when errors are localized.	They are **blunt instruments and may cause other disruptions to the service**. So while they may have broader impact than a precise solution, they can be put in place quickly to stop the bleeding while the team discovers and addresses the root cause. The bluntness is the price of not needing a diagnosis, not a defect.
Location beats detail	The rule that makes generic mitigation possible: **to mitigate an incident, you don't have to fully understand the details — you only need to know the location of the root cause.**	In the large worked case, the responders identified a plausible cause at 9:56 a.m. If they had then rolled all images back to a known good state, the incident would have been mitigated by **10 a.m.** Instead the bespoke rebuild was still running at 10:59 — 90% done, then forced to restart — and the fix landed at 11:59.	Being able to mitigate an outage **before its cause is fully understood** is crucial to running robust services at high availability. Note the ordering claim hidden here: knowing the location arrives much earlier than knowing the cause, and that gap is where the saved time lives.
Bespoke mitigation	A fix built for this specific fault, which therefore cannot start until the fault is understood.	In the same case: rebuilding binaries to push a new configuration that fetched images from a different location. The rebuild took about an hour; at 10:59, with the team 90% done, they discovered another location using the bad path and had to **restart the build**.	vs generic. Bespoke work is not wrong — it was one of two options assigned owners in that incident — but it is slow, and it is exposed to exactly the kind of setback above. The error is running it *instead of* a generic mitigation rather than alongside one.
Mitigation tools	The tooling that makes a generic mitigation a one-command action rather than a project.	The responders in that case would have benefited from some sort of tool that facilitated rollbacks.	**Mitigation tools do take engineering time to develop. The right time to create general-purpose mitigation tools is before an incident occurs, not when you are responding to an emergency.** Where to find out which ones you need: browsing postmortems is a great way to discover mitigations and tools that would have been useful in retrospect.
Mitigating is not preventing	Stopping the impact and stopping the recurrence are different achievements, and only the second needs the root cause.	In the mishandled case, mitigation successfully stopped the impact on **three separate occasions** — each time by raising a quota — but the team could only prevent the issue from recurring once they discovered the root cause.	The general policy is to **first stop the impact of an incident, and then find the root cause**, unless the root cause happens to be identified early on. Once mitigated, understanding the cause is *just as important*, to stop it happening again. What that case got wrong was continuing the rollout after the first mitigation: better to have postponed it until the root cause was fully determined.
Assessing impact	Step 1, and it is a verification rather than an assumption: confirm that real users are affected, and how.	In the large case the on-call checked a dashboard showing failures above **60% for two zones**, then verified the issue was affecting user attempts to create new clusters while traffic to existing ones was unaffected — and declared an incident **25 minutes after being paged**.	This is listed as something that **went well** in a case where much did not. It is also what licenses the outward communication: a notice went to users 18 minutes after the declaration, which is only possible because someone had established what to tell them.

Escalation paths and “all hands on deck”	Prepared routes to more people: documented escalation paths, an on-call developer who can be paged for emergencies, and a broadcast to prepared mailing lists.	In the large case the commander escalated to the infrastructure, networking and compute teams to eliminate them as causes; the communications lead then sent an “all hands on deck” email to several lists including all developer leads. A security-team engineer who joined that way found the failing component.	vs escalation as an admission of failure. The lists existed beforehand, which is what made the broadcast a thirty-second action. Escalation is not a sin — especially if it helps lower customer pain.
Heroics	Individuals rescuing an incident outside their working hours, off-rota, because they happened to be reachable.	In the mishandled case, developers rallied on a weekend and provided valuable input. This was **both good and bad**: the effort was valued, but **successful incident management shouldn't rely on heroic efforts of individuals**. What if they had been unreachable?	vs a staffing solution. The stated alternatives are structural: conduct rollouts during business hours, or organise an on-call rotation that provides **paid** coverage outside them. Note the related failure in the same case: the rollout continued over a weekend when developers were not readily available, against the practice of rolling out only on business days.
Delegating the defined, coordinating the undefined	The commander's allocation rule in the well-handled case: hand off the work that has procedures, and personally drive the work that has none.	Three objectives after a power loss. Two — safely restoring grid power, and restarting machines not hosting customer workloads — were **clearly defined, well understood, and documented**, and went to the operations lead with regular status reports back. The third, moving unaffected workloads off hosts that needed rebooting, was **more vague and wasn't covered by any existing procedures**.	For the third, the commander assigned a dedicated person to coordinate between the two teams, monitored progress closely, realised the work called for **new tools to be written quickly**, and organised more engineers to build them. The command role also stayed with the team that had **the best visibility into customer impact** — which is why, in that case, it stayed there.

CONCEPT BOUNDARY — “WE DON'T UNDERSTAND IT YET” IS NOT A REASON TO WAIT

The instinct: acting before you understand is reckless. You might make it worse, and you will certainly look as though you are guessing.

Why the instinct is wrong here: a generic mitigation is defined as an action taken *before the root cause is fully understood*. That is its purpose, not a compromise. It needs one input only — roughly where the problem is — and that input becomes available long before a cause does.

What is genuinely being traded: generic mitigations are blunt, and may cause other disruptions to the service. A rollback undoes good changes with the bad; draining a region moves load somewhere that may not want it. You accept a broader, smaller harm to end a narrower, larger one.

The line that settles it: customers do not care whether or not you fully understand what caused an outage. What they want is to stop receiving errors.

Anchor sketch — the four steps, and where the hour goes

Legend: bold = a named step or move **reversed = the step people postpone** **Q = cover the page and answer this**

```
AN ACTIVE INCIDENT, IN ORDER
-----------------------------------------------------
1  ASSESS THE IMPACT  verify real users are hurt,
      and how. this is what you can then announce.
2  MITIGATE THE IMPACT  <- the one that gets skipped
3  ROOT-CAUSE ANALYSIS
4  AFTER IT IS OVER  fix the cause, write the
      postmortem, then drill the vulnerability

MITIGATE FIRST -- the rule that makes step 2 possible:
     you do NOT need the details.
     you need the LOCATION of the root cause.
                       |
     +-- GENERIC MITIGATION . works with no diagnosis
     |     roll back a release correlated with the outage
     |     reconfigure load balancers away from a region
     |     PRICE: blunt. may disrupt other things. take it.
     |
     +-- BESPOKE MITIGATION . needs the diagnosis first
           rebuild + push a new config: about an hour,
           and it can be knocked back to zero

WHAT IT COST IN ONE REAL INCIDENT
-----------------------------------------------------
     7:00   impact assessed .............. users confirmed
     9:56   possible cause found ......... location known
     |          +--> a generic rollback HERE would have
     |               mitigated it by 10:00
    10:59   bespoke rebuild at 90% ....... a second bad
                                           path found ->
                                           restart build
    11:59   root cause found and fixed
    12:11   all zones back to 0% error
     total: 6 h 40 m of failure

MITIGATION IS NOT PREVENTION
     stopping the impact 3 times did not stop it coming
     back. only the ROOT CAUSE did.
     and: build the rollback tooling BEFORE the emergency.
     read old postmortems to find out which tools.
```

Q Give the four steps in order, and say which is top priority and why.

Q What is the single input a generic mitigation needs, and what does it *not* need?

Q Give two examples of a generic mitigation, and the price they both carry.

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN

Applying a generic mitigation as soon as you know the location	The bleeding stops, hours before a diagnosis would have arrived.	A blunt instrument's side effects — good changes rolled back with bad, load moved somewhere unwelcome.	As soon as step 1 is done and you know roughly where. Mitigation outranks everything.
Pursuing a bespoke fix	A precise repair with no collateral damage.	It cannot start until you understand the fault, takes far longer, and can be reset by a late discovery.	Alongside a generic mitigation, never instead of one.
Building mitigation tooling in advance	Turns a generic mitigation from a project into a command.	Engineering time, spent on something no incident is currently demanding.	Before the incident — and let old postmortems tell you which tools would have helped.
Continuing a rollout after mitigating	Momentum on the release.	The impact returns, because mitigation was never prevention — and in the worked case it returned at 100% rollout, on a weekend.	Not until the root cause is fully determined.
Relying on people who happen to be reachable	A resolution this weekend.	A response that depends on luck, and on unpaid time.	Never as a plan. Roll out in business hours, or pay for out-of-hours coverage.
Broadcasting “all hands on deck”	Expertise you did not know you needed — in the worked case, the person who found the failing component arrived this way.	Many people's attention, and a noisier channel.	When the incident is hours old with no cause and no mitigation. The lists must exist already.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“Let's find out what's causing it.”	“Let's stop it first. What can we roll back or drain right now, without knowing the cause?”
“We can't act until we understand it.”	“We don't need the details, only the location. We have that — so we can mitigate now and diagnose after.”
“A rollback might break other things.”	“It might. Generic mitigations are blunt by design; that's the price of not needing a diagnosis. It's cheaper than another two hours of errors.”
“The fix is nearly built.”	“Keep building it, and mitigate generically in parallel. Bespoke work can be knocked back to zero by one late discovery.”
“It's mitigated, so we're fine.”	“It's mitigated, not prevented. Hold the rollout until we have the root cause — this one has come back twice already.”
“We got lucky, people were around at the weekend.”	“That's the finding, not the happy ending. Either we roll out in business hours or we pay for out-of-hours cover.”
“We should write a rollback tool sometime.”	“Now, not during the next one. And the last three postmortems will tell us which tools we actually needed.”

PART 2 · QUESTION 1

A search service starts returning empty results for about a fifth of queries. Forty minutes in, the responders establish that every failing query is served from one of three sites in one continent, and that a configuration push went out to those sites shortly before the errors began — but they cannot yet say what in the configuration is wrong. The team lead's position is that pushing the previous configuration back would "undo a week of unrelated changes as well" and that they should keep reading the diff until they find the offending line. Two engineers are already writing a targeted patch, which they estimate at ninety minutes.

(a) Give the four steps of an active incident in order, and say which step this team is on and which one they have skipped. **(b)** State the rule about details versus location, and apply it: is enough known to act, and on what evidence? **(c)** Take the team lead's objection at full strength, name what it correctly identifies, and say why it does not change the decision. **(d)** The targeted patch is not wasted work. Say what should happen to it and why, referring to what went wrong with the equivalent effort in the worked case.

PART 2 · QUESTION 2

A queue-backed notification system has now overflowed three times in a fortnight. Each time, an engineer has raised the consumer's memory limit, the backlog has drained, and the alert has cleared; nobody has found out why the backlog forms. A fourth occurrence lands on a Sunday, during a staged rollout that is at 60% and still progressing. The engineer who fixed it the previous three times is on holiday but answers a message and talks a colleague through it.

(a) Name precisely what the three previous fixes achieved and what they did not. **(b)** Two decisions in this story are the same mistake as in the mishandled worked case. Name both, and give the specific corrective for each. **(c)** The holidaying engineer answered, so the incident was handled. Explain why this is recorded as a finding rather than a relief, and give the two structural alternatives. **(d)** Say what should be done with the rollout now, and what would have to be true before it resumes.

2×2 — what you know, and what you can reach for

COLUMNS: IS A GENERIC MITIGATION AVAILABLE TO YOU? →

	Yes — rollback, drain, failover, a switch	**No — only a bespoke fix exists**
You know roughly WHERE the cause is	**Act now — this is the whole lesson** Location is the only input a generic mitigation needs. Waiting for the details here is what cost the worked case its last two hours of impact. *Move:* apply it, accept the bluntness, and keep diagnosing behind it. In the worked case this cell was reached at 9:56 with no rollback tool ready — the gap the next cell describes — and a rollback then would have ended the impact by 10:00 instead of 12:11.	**The expensive cell** You could stop it, but nobody built the switch. Every minute now is spent on a fix that cannot ship until it is finished, and can be reset by a late discovery. *Move:* improvise the bluntest thing available — take the region out, stop the rollout, shed load — then put "build the tool" in the postmortem. Old postmortems are where you find out which tool you were missing.
You do NOT yet know where	**Aim the blunt instrument** You hold the lever but do not know which way to pull it. This is where step 1 earns its place: assessing impact usually localises the fault. *Move:* finish assessing — which users, which region, which version, since when — and correlate with recent change. That is a much shorter job than diagnosis, and it moves you up a row.	**Genuinely early** Nothing to pull and nowhere to aim. The only honest cell of the four, and the one people believe they are in for far longer than they actually are. *Move:* escalate wide, get people who know the system into one place, and keep a shared document of causes eliminated. Re-ask "do we know where yet?" every few minutes — the answer usually arrives well before the cause does.

Read it as: the row is what you know and it improves quickly; the column is what you built, and during an incident it cannot change at all. Most responses stall in the top-left, holding both the knowledge and the lever, because acting without a diagnosis feels like guessing — and that hesitation is the single most expensive habit in this competency.

CARRY-OUT RULE FROM THIS PART

Ask "can we stop this without knowing why?" before you ask "why?" Ask it out loud, and ask it again every ten minutes. The answer changes as the location becomes clearer, and the moment it turns into "yes" is the moment the incident should end for your users — regardless of how much of it you understand.

3

PART 3 OF 4

Practise before the pager

What to teach, what to decide in advance, and how to find the gaps while the stakes are low.

TENTH-GRADER VERSION

- Nobody reads the fire evacuation map during a fire. Everything that has to work under stress has to be familiar before the stress.
- Teach people the shape of a response — that they can delegate, that they can escalate, that mitigation comes first, and what the three roles are.
- Decide the boring logistics now: which chat channel, who to contact, what wording to publish, and what even counts as an incident.
- Tell people what's happening. If you don't say an incident is being worked on, everyone assumes nobody is working on it. If you don't say it's over, everyone assumes it isn't.
- Then practise. Invent an outage. Break something on purpose in a test environment. Treat a small problem as a big one just to run the drill.
- The valuable bit of a drill isn't the drill — it's the report afterwards saying what didn't work.
- Practice builds muscle memory, and muscle memory is what you have left when you panic.

THE EVERYDAY VERSION

Every building you have ever worked in runs fire drills, and everyone finds them mildly annoying. Notice what a drill is actually testing, though — almost none of it is the fire. It tests whether the alarm is audible in the far corner of the basement; whether the door that is always propped open is still propped open; whether anyone remembers that the assembly point moved; whether the person with the visitor list is the person who was in that day. These are logistics questions, decided long before, and the drill exists to find out which of those decisions have quietly rotted. The genuinely valuable part is not the walk down the stairs. It is the five minutes afterwards when someone says “the third-floor stairwell door was locked” — and then somebody unlocks it. A drill nobody reviews is exercise; a drill with a report is maintenance.

The terms, each with its look-alike

TERM	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Response training	Training responders to **organize** an incident, so they have a pattern to follow in a real emergency.	What it buys: knowing how to organise an incident, having a common language to use throughout, and sharing the same expectations — which together **reduce the chance of miscommunication**.	vs training people to fix things. This is training in the *shape* of a response, not in any system. Three suggested starting points if the full framework is more than you need: let on-calls know they can delegate and escalate; encourage a mitigation-first response; define the commander, communications and operations roles.

Connecting theory to scenarios	People are **more receptive to incident response training when they can connect the theory to actual scenarios and concrete actions**.	So include hands-on exercises, and share what happened in past incidents — analysing what went well and what didn't. You can adapt and summarise your framework into a deck for new team members, and external agencies specialising in incident response classes are an option.	vs presenting the framework as doctrine. The abstract version is what people forget; the story of an outage they recognise is what they retain.
Deciding the channel	Agree the communication channel beforehand — chat, a phone bridge, or similar — and **practise using it so there are no surprises**.	The reason, stated plainly: **no Incident Commander wants to make this decision during an incident.** If possible pick one the team is already familiar with, so everyone feels comfortable using it.	vs picking the best tool. Familiarity beats capability here, because the cost you are avoiding is hesitation, not missing features. One organisation's practice makes the split explicit: coordination decisions are made on a conference call and the outcomes are recorded in chat, with calls recorded so the timeline can be reconstructed if the scribe misses something.
Keeping your audience informed	Two symmetrical assumptions people make in the absence of information.	**Unless you acknowledge that an incident is happening and actively being addressed, people will automatically assume nothing is being done.** Similarly, **if you forget to call off the response once the issue is mitigated or resolved, people will assume the incident is ongoing.**	vs a communication task that ends when the fault does. Both failures are failures of the *edges* — the start and the end — and both are preempted by regular status updates plus a prepared contact list. Note that closing the incident is an announcement, not merely a state.
Prepared templates and contacts	Two or three ready-to-use templates for sharing information, which the on-call knows how to send; and a list of people to email or page, prepared beforehand.	The reason for templates: **no one wants to write these announcements under extreme stress with no guidelines.** Think ahead about how you will draft, review, approve and release public posts — one organisation's teams seek guidance from their public relations team.	The contact list is what makes a broadcast instant: in the large worked case, the “all hands on deck” call was an email to several lists that had been **prepared beforehand**. Both items save critical time and ensure you don't miss anyone.
Criteria for an incident	An established list of criteria for determining whether a paging issue is **indeed an incident**.	Sometimes it is clear; other times it is not. A team can come up with a solid list by **looking at past outages** and taking known high-risk areas into consideration.	vs judgement in the moment. This is the mechanism that makes “declare early and often” actionable rather than aspirational, because it removes the argument about whether the threshold has been met. The summary: decide how to communicate, who your audience is, and who is responsible for what — guidelines that are easy to set up and have high impact on shortening resolution time.

Drills	The final step: practising your incident management skills, so the team develops good habits and patterns of behaviour before they are needed.	Three scales. A company-wide resilience-testing programme creates a **controlled emergency that doesn't actually impact customers**, which teams respond to as if it were real and review afterwards. Smaller: a role-playing exercise rehearsing specific incidents. Smallest: **intentionally treating minor problems as major ones** requiring a large-scale response.	vs testing the system. Drills test the *response*. Everyone who could get swept in should feel comfortable with the tactics — including developers, customer support and marketing partners, not only engineers on call.
Running a drill well	Invent an outage, or build one from a postmortem — **postmortems contain plenty of ideas for drills**. Use real tools as much as possible; consider breaking your test environment so the team performs real troubleshooting.	Drills are **far more useful if they're run periodically**, and each should be followed by a report detailing what went well, what didn't, and how things could have been handled better.	**The most valuable part of running a drill is examining the outcomes**, which can reveal a lot about gaps in incident management — and once you know what they are you can work toward closing them, so a drill without a review has skipped its own point. What made one such programme succeed organisationally: accepting failure as a means of learning, finding value in the gaps identified, and getting leadership on board.
Simulation with a clock	Practice exercises teach roles and process, but **can't fully replicate the urgency of actual major incidents**. Time-bound simulation games are used to capture that aspect.	One weekly failure-injection exercise nominates a commander beforehand — typically a person training to become one — who is expected to behave and act like a real commander throughout, while subject-matter experts use the same processes and vernacular they would use during an actual incident.	The exercise serves both directions of experience: it familiarises new on-call engineers with the language and processes, and gives seasoned ones a refresher. The separate use of a cooperative, time-limited puzzle game is deliberate — its stressful, communication-intensive nature forces cooperation under a deadline, which calmer exercises cannot manufacture.
Rotating during a long incident	Swapping responders, including the commander, on a fixed interval during a long response.	In an incident that ran past ten hours, on-call engineers and the commander were rotated **every four hours**. When the runbook procedures did not resolve the issue, the response team started trying new recovery options **in a methodical manner**.	The two stated reasons are different in kind: it **encouraged engineers to get rest**, and it **brought new ideas into the response team**. The second is easy to miss — rotation is not only a welfare measure, it is a debugging technique.

CONCEPT BOUNDARY — PREPARATION IS NOT DOCUMENTATION

What organisations usually produce: a written policy, filed, linked from an onboarding page, technically correct and never read under stress.

What is actually being asked for: decisions made in advance and then *rehearsed* — a channel already used for this, templates the on-call has already sent, lists already mailed, criteria already argued over on a calm afternoon.

The test that separates them: a document tells you what you should do; practice determines what you will do. Staying calm and following a structure during an emergency **takes practice, and practice builds muscle memory** — which is what gives you the confidence you need in a real emergency.

The consequence for drills: a drill with no report afterwards is exercise, not preparation. Examining the outcomes is where the gaps become visible, and the gaps are the product.

Anchor sketch — everything that must be settled before the pager

Legend: bold = a named preparation **reversed = what turns the rest into reflex** **Q = cover the page and answer this**

```
"by the time a pager alerts you to a problem, it is too
 late to think about how to MANAGE the incident."
                       |
TRAIN  teach the SHAPE of a response, not a system
     you may delegate . you may escalate .
     mitigation comes first . the three roles exist
     +--> the full framework may be more than you need.
          take the parts that matter to you.
     +--> people take it in when it is attached to a
          REAL past incident. use hands-on exercises.
                       |
DECIDE THE LOGISTICS  all of it, on a calm day
     CHANNEL ....... agree it and PRACTISE it. pick one the
                     team knows. no commander wants to be
                     choosing a tool mid-incident.
     TEMPLATES ..... 2-3 ready to send, and the on-call knows
                     how. nobody writes well under stress.
     CONTACTS ...... prepared lists = an "all hands on deck"
                     broadcast takes 30 seconds.
     CRITERIA ...... what counts as an incident. build it from
                     PAST OUTAGES + known high-risk areas.
                     this makes "declare early" a rule, not
                     a mood.
                       |
TELL PEOPLE  two symmetrical assumptions:
     say nothing -> they assume NOBODY IS WORKING ON IT
     forget to call it off -> they assume IT IS STILL GOING
                       |
DRILL  practice builds muscle memory. muscle memory
is what is left when you panic.
     big ..... a controlled emergency, no real customer
               impact, treated as if it were real
     medium .. role-play a previous postmortem
     small ... treat a minor problem as a major one ON
               PURPOSE, just to run the machinery
     rules: run them PERIODICALLY . use real tools .
            consider breaking the test environment
     +--> the value is NOT the drill. it is the report after:
          what went well, what did not, what would be done
          differently. no report, no preparation.
     +--> long real incident? rotate responders AND the
          commander (one team: every 4 hours) -- for rest,
          and for FRESH IDEAS.
```

Q Name the four things to settle in advance, and the reason given for each.

Q Give the two symmetrical assumptions people make when you don't tell them anything.

Q What is the most valuable part of a drill, and what makes a drill worthless?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN

Adopting only part of the framework	Something a small team will actually use, taught in an afternoon.	Gaps the full structure would have covered, found later and the hard way.	Starting out — take delegation, escalation, mitigation-first and the three roles, and grow.
Picking the channel the team already uses	No hesitation, no learning curve, no surprises mid-incident.	It may be worse at the job than a purpose-built tool.	Almost always. Familiarity is the feature you are buying.
Writing announcement templates in advance	The on-call can inform the world without composing prose under stress.	Time spent, plus review and approval routes to agree while nothing is wrong.	Before you need them — two or three, and make sure the on-call knows how to send them.
Publishing criteria for what counts as an incident	Removes the argument about the threshold, which is what actually delays declaration.	Effort to derive them, and they must be revisited as the system changes.	As soon as you have past outages to learn from — which is the stated source for them.
Running a company-scale controlled emergency	Finds gaps nothing smaller can, across teams that never rehearse together.	Expensive, disruptive, and needs leadership backing to survive its first bad result.	When the organisation can accept failure as a means of learning — that acceptance is a precondition, not a benefit.
Treating a minor problem as major on purpose	Real tools, real people, real procedures, low stakes.	Attention spent on something that did not need a full response.	Regularly, as the cheapest drill available.
Rotating the commander every few hours	Rested responders, and fresh ideas entering a stuck investigation.	A handover, and the context that goes with it.	Any incident long enough that people are tiring — one worked case rotated every four hours across ten.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“We've written the incident policy.”	“Has anyone run it? A document tells people what they should do; only practice decides what they will do.”
“We'll set up a channel when something happens.”	“Agree it now and practise in it. No commander wants to be choosing a tool mid-incident.”
“Support can explain it to customers.”	“Give them two or three approved templates and make sure the on-call can send them. Nobody writes well under extreme stress.”
“It's obvious when something is an incident.”	“Sometimes. Write the criteria from our past outages and known high-risk areas — that's what makes declaring early actually happen.”
“We fixed it, we can stop updating now.”	“Call it off explicitly. If we don't say it's over, everyone assumes it's still going.”
“The drill went fine.”	“Then write the report. What went well, what didn't, what we'd do differently — the outcomes are the product, not the exercise.”

“She's been on it eight hours, she knows it best.”	“Rotate her out. It's for rest, and it's also how a stuck investigation gets a new idea.”

PART 3 · QUESTION 1

A sixty-person engineering organisation has a well-written incident response policy on its internal wiki, revised last year. During a genuine outage last month: the responders spent eleven minutes deciding between two chat tools and a video call; the on-call engineer wrote a customer-facing status update from scratch, which a manager then rewrote twice before it was published forty minutes later; nobody could find the escalation contact for the payments team; and two engineers argued for a quarter of an hour about whether the event qualified as an incident at all. The postmortem's single action item reads “re-read the incident policy”.

(a) Name the four things that should have been settled in advance, matching each to the specific delay it would have removed here. **(b)** For the channel and the templates, give the stated reason each must be decided beforehand rather than chosen well in the moment. **(c)** Explain what is wrong with the action item, using the distinction between documentation and preparation. **(d)** The criteria argument is the most consequential of the four delays. Say why, and say where the criteria should come from.

PART 3 · QUESTION 2

A team wants to start practising incident response but has no budget for anything elaborate. They propose a quarterly exercise in which the on-call engineer is given a written scenario and talks through what they would do, in a meeting, for twenty minutes. There is no report afterwards — the group discusses it and moves on. Separately, during a real nine-hour incident last week, the same commander ran the whole response because “she had all the context”, and by hour seven the team was re-testing hypotheses it had already eliminated.

(a) Give the three scales of drill available, and say which of them this team could run this week at no cost. **(b)** Name what their proposed exercise is missing that would make it preparation rather than conversation, and say what specifically that missing piece produces. **(c)** Their scenario is invented from scratch each time. Give the better source and say why. **(d)** Address the nine-hour incident: name the practice that was skipped, give both of its stated benefits, and say which one would have prevented the repeated hypotheses.

2×2 — decided in advance, and rehearsed

COLUMNS: HAS THE TEAM ACTUALLY PRACTISED? →

	Yes — drills, run periodically, reviewed	**No — they have been told, not trained**
The logistics ARE settled in advance	**Ready** Channel agreed and used, templates written and sendable, contacts listed, criteria published — and people who have run the machinery with nothing at stake. *Move:* keep drilling periodically and keep writing the report each time, because this cell decays quietly: contact lists rot, the channel gets replaced, the criteria stop matching the system. The drill is what detects the rot.	**A good policy, untested** Everything is decided and written down, and nobody has ever done it. The gaps are all still there; they are simply invisible. *Move:* the cheapest drill there is — take the next minor problem and deliberately run it as a major one, with real tools, real roles and a real report. It costs nothing and tests the decisions you have already made.
Nothing is settled — it gets decided on the night	**Skilled improvisers** People know how to run a response and burn the first fifteen minutes of every one choosing tools, writing prose and arguing about whether this counts. *Move:* write the four decisions down this week. This team gets the most immediate return of the four cells, because the skill is already there and only the friction is missing.	**Where every bad case in this booklet begins** No channel, no templates, no contacts, no criteria, no practice. The response will be assembled from scratch, by frightened people, at the worst moment. *Move:* start with criteria and channel — they are free and they unblock declaring. Then one drill, then the report. Do not attempt the full framework; take the parts that matter and grow into the rest.

Read it as: the row is cheap and can be done in an afternoon; the column is not, and cannot be bought late. Note that the two off-diagonal cells fail differently — the top-right has answers nobody has tested, the bottom-left has skill with no scaffolding — and the fix for each is the other one's strength.

CARRY-OUT RULE FROM THIS PART

Every decision you make during an incident is one you failed to make before it. Keep a running list of them: each time a response stalls to choose a tool, find a phone number, compose a sentence or argue about a threshold, that is next week's preparation task. The list writes itself, for free, from every incident and every drill — provided somebody writes the report afterwards.

4

PART 4 OF 4

The write-up that changes something

Two postmortems of the same outage — one that taught nothing, and one that changed the system.

TENTH-GRADER VERSION

- After it's over, someone writes up what happened. Most of the value of the whole incident is in that document, and most write-ups waste it.
- Don't blame people. Not because it's unkind — because blamed people go quiet, and the next write-up is missing the bit you needed.
- Use numbers. If you can't measure the damage, you can't know when it's fixed.
- “Improve the alerting” is not an action. “Alert when more than X% of our machines disappear” is, because you can tell whether it's done.
- Every action needs one owner, a priority, and a ticket. Without a ticket it will be forgotten.
- Don't try to fix humans. Fix the system that let the humans do it.
- Publish it widely and publish it fast. A write-up read by one team, four months late, has taught nobody anything.

THE EVERYDAY VERSION

Two documents can follow the same plane landing badly. One is a disciplinary hearing: who was flying, what they failed to do, what will happen to them. The other is an accident investigation: what the aircraft did, second by second, with the figures; which warnings were displayed and which were ambiguous; why a reasonable crew would have read them the way this crew did; and a numbered list of changes to the checklist, the instrument and the training, each with an owner and a deadline. Both are thorough. Only one of them is read by other airlines, and only one of them changes anything, and the reason is the same in both cases. The hearing tells everyone that the safe move is to say as little as possible; the investigation tells everyone that the useful move is to say exactly what happened. Every rule in this part is downstream of that single difference.

The worked case, in four sentences

A rack of edge proxy machines was marked for decommissioning, and the disk-wipe step of the decommission — which overwrites the full content of all drives, irreversibly — finished successfully for that rack, while the automation for the remaining steps failed. To debug the failure, an engineer retried the process. On the second run the step that fetched the machines in the rack returned an **empty list**, because they had already been decommissioned, and an **input validation bug** in the admin server treated an empty list as “no filter” rather than “act on no machines”. Within minutes the disks of all such machines **globally** were erased; user traffic was rerouted to core datacentres, causing a latency increase; it took **two days** to reinstall the machines, and several weeks afterwards to audit the automation and make the workflow idempotent.

Two postmortems of that outage are compared below. Both are about the same 40 minutes. One was published **four months** later to a single team; the other was published **four days** later to everyone in engineering. Three years on, a similar incident occurred — and the action items from the second document had **dramatically reduced the blast radius and rate** of the repeat.

The terms, each with its look-alike

FAILURE OR QUALITY	PLAIN DEFINITION	CONCRETE EXAMPLE	BOUNDARY — WHAT IT GETS CONFUSED WITH
Missing context	Introducing terminology specific to one system without explaining it, while the Background and Glossary sections sit empty.	If you need to provide additional context, use those sections — which can link to longer documents.	The consequence is not pedantry: an uncontextualised document **might be misunderstood or even ignored**. And the reason it matters is the audience rule — **your audience extends beyond the immediate team.**
Key details omitted	Sections that contain high-level summaries but lack the details that make them useful.	Three places it showed: a **problem summary** whose only numerical data was the duration, so the size of the outage cannot be estimated; a **root causes and trigger** section that describes the cause in a paragraph without exploring the lower-level details; and a **recovery efforts** section left completely empty.	On the missing numbers: for outages affecting multiple services you should present numbers to give a consistent representation of impact, and **even if there is no concrete data, a well-informed estimate is better than no data at all** — because **if you don't know how to measure it, then you can't know it's fixed.** On recovery efforts: the postmortem *is* the record of the incident for its readers, and that is the section which tells them how it was mitigated.
Weak action items	The section that is supposed to be an actionable plan to prevent recurrence, and instead is a list of intentions.	Four defects, all present at once: the items were **mostly mitigative** with almost no preventative ones; **all tagged with equal priority**, so there is no way to determine which to tackle first; two used **ambiguous phrases** like “Improve” and “Make better”, which are open to interpretation and make success criteria unmeasurable; and **only one had a tracking bug**.	The one “preventative” item proposed making humans less error-prone. In general, **trying to change human behavior is less reliable than changing automated systems and processes** — or, as one engineer put it, “let's plan for a future where we're all as stupid as we are today”. On tracking: **without a formal tracking process, action items from postmortems are often forgotten, resulting in outages.**
Finger pointing	Naming individuals as causes. Every postmortem has the potential to lapse into a blameful narrative.	It appeared in three separate sections of the bad example: the whole team blamed with two members called out by name; a named individual targeted in an action item; and one person **held solely responsible** in the root cause section.	vs accountability, which is what ownership provides. The stated mechanism of harm is specific: highlighting individuals **leads team members to become risk-averse because they're afraid of being publicly shamed**, and **they may be motivated to cover up facts critical to understanding and preventing recurrence**. The blameless version focuses on the **gaps in system design** that permitted the failure mode — on *what* went wrong, not *who* — and aims its actions at improving the system rather than improving people.
Animated language	Personal judgements and subjective language in a document that should be a **factual artifact**, considering multiple perspectives and respectful of others.	Three examples from the same document: superfluous language (“careless ignorance”), animated text (“which is ridiculous”), and an exclamation of disbelief (“I can't believe we survived this one!!!”).	The cost is twofold: dramatic descriptions **distract from the key message** and **erode psychological safety**. The replacement is not blandness — it is evidence: **provide verifiable data to justify the severity of a statement.**

Ownership	Declaring official ownership **results in accountability, which leads to action**.	Two failures: the document listed **four owners**, and the action items had little or no ownership.	Ideally an owner is a **single point of contact** responsible for the postmortem, the follow-up and the completion — **better to have a single owner and multiple collaborators**. Action items without clear owners are less likely to be resolved. Note this is the exact opposite of finger pointing: naming who *will act* is required; naming who *caused it* is forbidden.
Audience	By default the document should be accessible to **everyone at the company**; the recommendation is to share it as widely as possible — perhaps even with your customers.	The bad example was shared only among members of the team; the good one went to all engineering employees.	The principle behind it: **the value of a postmortem is proportional to the learning it creates**, and the more people who can learn from past incidents, the less likely they are to be repeated. A thoughtful and honest postmortem is also **a key tool in restoring shaken trust**. As a culture matures the audience may extend to non-humans, with machine-readable tags and metadata enabling downstream analytics.
Promptness	Writing and circulating the document while the incident is fresh.	The bad example was published **four months** after the incident. Had it recurred in that interval — and this outage did in reality recur — team members would likely have forgotten key details a timely document would have captured. The good one was circulated **less than a week** after the incident closed.	Two reasons, and the second is the one people miss. A prompt postmortem **tends to be more accurate because information is fresh**. And the people affected are waiting for an explanation and some demonstration that you have things under control: **the longer you wait, the more they will fill the gap with the products of their imagination.**
Depth, metrics and conciseness	Three of the qualities that made the second document work.	**Depth**: rather than only investigating the proximate area of failure, it explored impact and system flaws across multiple teams, with impact described from several perspectives and all conclusions based on facts and linked data. **Quantifiable metrics**: cache hit ratios, traffic levels and durations, with links back to original sources — data transparency that removes ambiguity. **Conciseness**: lengthy chat transcripts and system logs were abstracted, with the unedited versions linked.	Add **clarity**: a well-written glossary makes the document accessible to a broad audience, and grouping action items by theme makes it easier to assign owners and priorities — that incident's items were split into five themes covering prevention, emergency response, monitoring, provisioning and cleanup. Measurability is what makes an action item finishable: “add an alert when more than X% of our machines have been taken away from us” has a verifiable end state.

The culture that produces them	Postmortems are as much a cultural change as a technical one, and the culture has to be incentivised and defended.	**Rewarding:** reward action item *closeout*, not only authorship — reward writing but not closing and you risk an unvirtuous cycle of unclosed postmortems. Reward the organisational change with bonuses, reviews and promotion; highlight the improved reliability; hold authors up as experts; and gamify, with scoreboards or burndowns of open action items. **Sharing:** announce drafts organisation-wide, run cross-team reviews and reading clubs, reenact past incidents as training with the original commander present, and publish a weekly outage report.	**The four signs it is failing:** people *avoiding association* with a postmortem, and being glad not to be involved; leadership *failing to reinforce* — a "this is a safe space, but someone must have known" framing, which a responder can redirect by asking generically whether there were warning signs and why they were dismissed; *lacking time to write them*, since subpar postmortems with incomplete action items make recurrence far more likely — **postmortems are letters you write to future team members**, so keep a consistent quality bar; and *repeating incidents*, which should trigger questions about whether action items take too long to close, whether feature velocity is trumping reliability fixes, whether the right items are being captured, whether the service is overdue for a refactor, and whether people are putting sticking plasters on a more serious problem.

CONCEPT BOUNDARY — BLAMELESS IS NOT OWNERLESS, AND NOT VAGUE

The misreading: "blameless" means avoiding names, softening findings, and declining to say anything anyone might take personally. Documents written this way are gentle and useless.
What it actually forbids: attributing the *cause* to a person — holding someone solely responsible, calling out individuals for what went poorly, aiming an action item at a named person's behaviour. That is what makes people risk-averse and motivates them to cover up the facts you needed.
What it actively requires: naming a *single* owner of the document, and an owner plus a priority plus a tracking bug for every action item — because declaring official ownership results in accountability, which leads to action. Naming who will act is mandatory; naming who caused it is forbidden. Those are not in tension; they are the same instinct applied to the future rather than the past.
The test: read every sentence and ask whether it is about a *gap in the system* or about a *person's judgement*. Then read every action item and ask whether you could tell, six months from now, whether it is done.

Anchor sketch — the same outage, written twice

Legend: bold = a named failure or quality **reversed = what costs you the next one** **Q = cover the page and answer this**

```
ONE OUTAGE. 40 MINUTES. TWO DOCUMENTS.
------------------------------------------------------
THE WRITE-UP THAT TAUGHT NOTHING
     MISSING CONTEXT ..... jargon, and the Background
          and Glossary sections both left blank
     DETAILS OMITTED ..... the only number was the
          DURATION. recovery efforts: empty.
          "if you don't know how to measure it, then
           you can't know it's fixed"
     WEAK ACTIONS ........ "Improve alerting."
          "Make automation better."
          all one priority . 1 of 4 had a ticket .
          mostly mitigate, almost no prevent .
          the one "prevent" tried to fix HUMANS
     FINGER POINTING ..... a team blamed, two named,
          one "solely responsible"
          +--> people go risk-averse, and start
               COVERING UP the facts you needed
     ANIMATED LANGUAGE ... "careless ignorance"
          "which is ridiculous" / "I can't believe
          we survived this one!!!"
          +--> distracts, and erodes psychological safety
     NO REAL OWNER ....... four owners = none
     TINY AUDIENCE ....... one team
     FOUR MONTHS LATE .... details go stale; and this
                           outage did recur

THE WRITE-UP THAT CHANGED THE SYSTEM
     CLARITY ........ glossary . actions grouped into
                      five themes
     METRICS ........ real figures, linked to sources
     ACTIONS ........ every one has an OWNER, a
                      PRIORITY and a TICKET, and a
                      verifiable end state
                      ("alert when more than X% of our
                       machines have been taken away")
     BLAMELESS ...... what went wrong, not who
     DEPTH .......... impact across teams, not just the
                      proximate failure
     PROMPT ......... circulated in under a week
                      "the longer you wait, the more
                       they will fill the gap with the
                       products of their imagination"
     CONCISE ........ long logs abstracted, originals
                      linked
     WIDE ........... all of engineering
------------------------------------------------------
THREE YEARS LATER a similar incident happened. the
action items from the second document had DRAMATICALLY
REDUCED the blast radius and the rate.
```

Q Give four defects of a weak action-item list, and the rule about fixing humans.

Q State the mechanism by which blame destroys the *next* postmortem.

Q Why is publishing promptly about accuracy *and* about the audience?

Trade-offs

DECISION	BUYS YOU	COSTS YOU	CHOOSE IT WHEN
Publishing within days rather than when it is complete	Accuracy while memories are fresh, and an audience that stops inventing its own explanation.	Sections you would rather have finished, and figures still marked as estimates.	Always — a well-informed estimate is better than no data at all.
Sharing company-wide by default	Learning proportional to the readership, and a tool for restoring shaken trust.	Exposure: your team's worst day, readable by everyone.	By default — and consider going further, to customers.
One owner instead of four	A single point of contact accountable for follow-up and completion.	The appearance of pinning a whole team's work on one person.	Always. Multiple collaborators, one owner.
Preventative action items over mitigative ones	The outage stops being possible rather than becoming survivable.	They are slower, more invasive, and harder to get scheduled.	Every theme needs some — a list that is all mitigation has not addressed recurrence.
Grouping action items by theme	Owners and priorities become easy to assign, and coverage gaps become visible.	Overhead on a small incident with three items.	Large incidents — the worked case used five themes.
Rewarding closeout, not just authorship	Action items that actually close.	Recognition arriving months after the writing, when attention has moved on.	Always — rewarding only the writing risks an unvirtuous cycle of unclosed postmortems.
Requiring a P0 or P1 bug for any user-affecting outage	A guarantee that something is tracked, and a hard floor under the whole practice.	Rigidity, and pressure to file something rather than nothing.	When you accept the premise: to users, a postmortem without subsequent action is indistinguishable from no postmortem.

Say it so you understand it → say it like an expert

SAY IT THIS WAY TO *UNDERSTAND* IT	SAY IT THIS WAY TO SOUND LIKE AN *EXPERT*
“He ran it twice, that's what did it.”	“The system accepted an empty filter and treated it as ‘everything’. That's the finding. Who typed it is not.”
“Action item: improve the alerting.”	“Alert when more than X% of our machines are taken away. Owner, priority, ticket — otherwise I can't tell if it's done.”
“We'll train people not to do that.”	“Changing human behaviour is less reliable than changing the system. What check would have rejected the command?”
“We don't have exact numbers yet.”	“Put in a well-informed estimate and say it's one. If we can't measure the impact we can't know when it's fixed.”
“I'll publish it once every section is complete.”	“Publish this week. Every day we wait, people fill the gap with the products of their imagination — and details go stale.”
“It's a bit sensitive, let's keep it in the team.”	“The value is proportional to the learning it creates. Company-wide by default; an honest one is how we get trust back.”

“We wrote it up, so we're done.”	“To our users, a postmortem without subsequent action is indistinguishable from no postmortem. Who owns each item, and by when?”
“That team keeps having the same outage.”	“Then ask the five questions: are the items closing, is velocity beating reliability, are we capturing the right items, is the service overdue a refactor, are we patching a deeper problem?”

PART 4 · QUESTION 1

Here is the whole Lessons Learned section of a postmortem for a six-hour billing outage. *“Things that went poorly: nobody on the platform team had documented the failover procedure, which is frankly astonishing for a service this old. Priya was on call and did not escalate for two hours. Where we got lucky: the batch job hadn't started yet — I don't want to think about what would have happened otherwise!”* The action items are: “Make failover more reliable” (P2, no owner, no ticket); “Improve on-call training so people escalate sooner” (P2, no owner, no ticket); “Document the failover runbook” (P2, owner: Priya, ticket BUG-4471). The document lists five owners and was shared with the platform team.

(a) Identify every distinct failure mode present, naming each. **(b)** For the blame, state the mechanism by which it damages the organisation — and say why that mechanism makes this a practical problem rather than an etiquette one. **(c)** Rewrite the three action items so they satisfy every stated characteristic of a good one, and say which one you would delete outright and why. **(d)** Two structural facts are given at the end. Name what is wrong with each and what it should be instead.

PART 4 · QUESTION 2

A director notices three things over six months. Postmortems for that org are consistently published seven to ten weeks after the incident. In a review meeting a vice-president said, “Look, blameless is fine in principle, but someone in that room clearly knew this was risky — why did nobody speak up?” And a team has now had three outages with near-identical causes, each with a completed postmortem and every action item closed.

(a) For the publication delay, give both harms, and name the underlying organisational cause along with the rule about quality bars. **(b)** Give a specific reply to the vice-president that redirects the conversation without contradicting them, and state the principle about good faith and information that justifies it. **(c)** The third observation is the strangest: closed action items and recurring outages. Give the questions to ask, and say what it means when the items are all closing on time. **(d)** Name two incentives the director could introduce that target closeout and organisational learning rather than authorship, and say what each is designed to prevent.

2×2 — is it safe to write, and is it possible to act on

COLUMNS: CAN THE ACTIONS ACTUALLY BE FINISHED? →

	Yes — owner, priority, ticket, verifiable end state	No — vague, unowned, unprioritised, untracked
BLAMELESS — findings are about the system	**The document that changes something** People told the truth because it was safe to, and the truth was turned into work somebody owns. *Move:* publish it widely and quickly, group the items by theme, and reward the closing rather than the writing. This is the cell where a repeat of the same outage arrives smaller and rarer — which is the only evidence any of this works.	**Kind, thorough, and inert** Nobody is blamed, everybody agrees, and in a year nothing has changed. “Improve the alerting” can never be finished, so it never is. *Move:* rewrite each item until you could tell from the outside whether it is done, then give it one owner, a priority and a ticket. Without a formal tracking process, action items are simply forgotten — and that is how the outage returns.
BLAMEFUL — findings are about people	**It works once** The actions are sharp and they will get done. The cost is deferred and lands on the *next* incident, when people are careful about what they write down. *Move:* strip the names from the causes, keep them on the actions. If you cannot see the difference, that is exactly the confusion this quadrant is made of.	**Worse than writing nothing** Individuals named as causes, actions aimed at making humans less error-prone, nothing owned or tracked. It teaches the organisation that postmortems are where blame is assigned. *Move:* stop, and rewrite from the system's behaviour outward. Ask what check would have rejected the action — changing systems and processes is more reliable than changing people.

Read it as: the row decides whether you will get the truth *next* time, and the column decides whether anything changes *this* time. The two are independent, which is why a document can be scrupulously fair and completely useless, or sharply actionable and quietly corrosive — and why judging a postmortem by how thorough it feels tells you nothing about which cell it is in.

CARRY-OUT RULE FROM THIS PART

Read your own action items as a stranger, six months late. For each one ask two questions: can I tell whether this is done, and who exactly would I ask? If either answer is missing, the item will not close — and to your users, a postmortem without subsequent action is indistinguishable from no postmortem.

SECTION 6

Concept sketch — the whole competency, end to end

Four named groups, in the order the story runs: **STRUCTURE → ORDER → PREPARATION → THE RECORD**. Every node carries a question. Cover the answers, walk the map, and each answer hands you the next question.

Legend: bold = a group heading **reversed = the one group to remember** **Q = retrieval cue**

The four groups run in order: **STRUCTURE** — who is in charge · **ORDER** — what to do first · **PREPARATION** — decided in advance · **THE RECORD** — what outlives it.

GROUP 1 - STRUCTURE . an outage is a coordination problem

RESOLVING is mitigating impact and/or restoring the service. MANAGING is coordinating the teams and keeping communication flowing BOTH ways.
Q which of the two has no owner by default?

DECLARE EARLY AND OFTEN. whoever declares it typically becomes the commander.
Q name the three things declaring early buys you

THE THREE Cs coordinate . communicate . control

THE THREE ROLES commander (holds everything not yet delegated) . operations (applies the tools) . communications (the public face)
Q where is the culprit when a response goes wrong?
Q which role can be subsumed back, and when?

ONE PLACE, real time. 3+ people -> a shared document of theories and eliminated causes.
Q what goes in the doc, what stays in the channel?

GROUP 2 - ORDER . mitigate first, understand later

---

1 ASSESS . 2 MITIGATE . 3 ROOT CAUSE . 4 FIX + WRITE UP

YOU NEED THE LOCATION, NOT THE DETAILS
Q which step is top priority, and what if it isn't?

GENERIC MITIGATION roll back . drain a region. blunt, may disrupt other things - take it anyway. build the tooling BEFORE; old postmortems say which was missing.
Q in the worked case, how many hours did hesitating cost?

MITIGATION IS NOT PREVENTION. three mitigations did not stop it returning; only the root cause did.

NO HEROICS . business-hours rollouts, or PAID cover
Q so what should have happened to the rollout?
Q why is a weekend rescue a finding, not a happy ending?

GROUP 3 - PREPARATION . decided calmly, or not at all

TRAIN the shape: you may delegate . you may escalate . mitigation first . the three roles exist

SETTLE FOUR THINGS channel . templates . contacts . criteria for what counts as an incident
Q where do the criteria come from?

TELL PEOPLE. silence at the start = "nobody is on it". silence at the end = "it is still going".
Q so what is closing an incident, procedurally?

DRILL controlled emergency . role-play a postmortem . treat a small thing as big ON PURPOSE

```
        Q what is a drill's actual product?
   ROTATE on long incidents - for rest AND for fresh ideas
        Q which of those two is the debugging technique?

GROUP 4 - THE RECORD . the only part that outlives it
-----------------------------------------------------------------
   BLAMELESS  what went wrong, never who. blame makes people
        risk-averse and motivates them to COVER UP facts.
   BUT OWNED  one owner for the document; an owner, a priority
        and a ticket for every action
        Q why is blame a practical problem, not a courtesy?
        Q reconcile "no names" with "name an owner"
   ACTIONABLE  verifiable end state . preventative, not only
        mitigative . don't fix humans . MEASURED, with numbers
        or an estimate: cannot measure -> cannot know it is fixed
        Q rewrite "improve the alerting" so it can be finished
   PROMPT + WIDE  under a week, whole company. wait, and people
        fill the gap with their imagination.
        Q what did the good write-up do to the repeat incident?
```

SECTION 6 CONTINUED · THE TRANSFERABLE CARD

Master 2×2 — the decision card you can carry to other problems

Two questions, asked at opposite ends of an emergency.

COLUMNS: AFTERWARDS, DID ANYTHING IN THE SYSTEM CHANGE? →

	Yes — owned, tracked changes that closed	**No — a document, and good intentions**
DURING: one named person was in command	**Handled, and learned from** Someone coordinated while it burned, and something was different afterwards. The only cell where a repeat arrives smaller and rarer. *Move:* protect both halves; they decay for different reasons — command when declaring feels like over-reacting, change when nobody rewards a closure.	**A good war story** The response was crisp, the write-up reads well — and nothing is different, so the same event is scheduled to happen again. *Move:* go item by item and ask whether a stranger could tell it was done, and who they would ask. An action-free write-up is indistinguishable from no write-up.
DURING: everyone helped; nobody led	**Fixed by luck, then fixed properly** The response was chaotic — duplicated work, an audience left guessing — but the review was honest enough to change the system. *Move:* the changes will hold. Now fix the response, which is cheap: criteria for declaring, a named commander, one channel.	**The loop that never closes** Nobody led, nothing changed, and the next one will be handled exactly as badly. *Move:* pick the cheapest end. Declaring costs one sentence; giving three action items owners and tickets costs an afternoon. Take the parts that matter and grow into the rest.

Read it as: the row is decided in the first minute, the column over the following weeks, and neither substitutes for the other. The same pair separates a serious incident review from a disciplinary hearing, an accident report from a verdict, and a safeguarding review from a meeting where everyone agreed it was unfortunate.

THE THUMB RULE, IN ONE SENTENCE

Somebody has to be in charge while it burns, and something has to be different once it's out.

Both are cheap, and both are skipped for opposite reasons: command because declaring feels like over-reacting, change because the crisis is over.

SECTION 7

Symptom → diagnosis → move

Cover the middle and right columns and work down the left.

During an incident

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
Several people are investigating and you cannot say who is in charge.	No incident was declared, so no command line exists.	Declare, take command, and name the two roles. Whoever declares it typically becomes commander; they hold every role not yet delegated.
Two responders have eliminated the same cause independently.	No working record — the coordination is happening in people's heads.	Start the shared document: working theories, eliminated causes, error logs, suspect graphs. Three or more responders is the threshold.
The commander is answering a director, debugging, and drafting an update.	Nothing has been delegated, so all three roles are still theirs.	Delegate operations, delegate communications, or hand over command and take operations. All three are normal moves.
Support keeps interrupting the people fixing it.	No communications lead.	Appoint one. They give periodic updates to the team and stakeholders and absorb the inquiries; the role collapses back into command when the incident shrinks.
The channel has a dozen participants and is unreadable.	Detail and decisions are competing for the same space.	Move detailed debugging to the shared document and keep the channel as the decision hub.
Coordination is happening in the bug tracker.	Communication was never centralised.	Get everyone into one place — room, channel or call — talking in real time. A tracker tracks; it does not coordinate.
Hours in, with no cause and no mitigation.	The response has run out of expertise, not effort.	Escalate wide. Page the neighbouring on-calls to eliminate their areas, and broadcast to prepared lists. Escalation is not a sin, especially if it lowers customer pain.
You know which region and which release, but not what is wrong.	You already have everything a generic mitigation needs.	Roll back or drain, now. You need the location of the cause, not its details.
“We shouldn't roll back, it'll undo unrelated changes.”	The bluntness of a generic mitigation, correctly identified.	Accept it. Blunt is the price of not needing a diagnosis; the impact ends now and the precise fix continues behind it.
The targeted fix is nearly built.	Bespoke work being run instead of a mitigation.	Run both. In the worked case a rebuild reached 90% and had to restart when a second bad path appeared.
The alert cleared, so the incident is over.	Impact mitigated; recurrence untouched.	Keep going to root cause, and hold any in-flight rollout until you have it.
The response is being carried by someone who is off-rota.	Heroics standing in for staffing.	Note it as a finding. Roll out in business hours, or pay for out-of-hours coverage.
Hour seven, and the team is re-testing an eliminated hypothesis.	Fatigue, and no fresh perspective entering.	Rotate responders and the commander. It is for rest *and* for new ideas; the shared document guards the same failure from the other side.
Nobody outside the response knows anything.	The outward half of managing an incident is not being done.	Give regular status updates. Silence is read as “nobody is working on it”.
It is fixed, and people are still asking about it.	The response was never called off.	Announce the close explicitly, and validate recovery with each on-call before you do.

Preparing, between incidents

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
The first ten minutes of every incident go on choosing a tool.	The channel was never agreed.	Agree it and practise in it. Prefer one the team already knows — no commander wants to choose mid-incident.
A status update took forty minutes and three drafts.	No templates.	Write two or three ready-to-use ones, agree the review and approval route, and make sure the on-call knows how to send them.
Nobody could find the escalation contact for another team.	No prepared contact list.	Build it. It is what turns “all hands on deck” into a thirty-second action.
People argue about whether something counts as an incident.	No published criteria.	Derive them from past outages and known high-risk areas. This is what makes “declare early” operable.
The policy is excellent and nobody has ever run it.	Documentation mistaken for preparation.	Drill. A document says what people should do; practice decides what they will do, and practice is what builds muscle memory.
Drills happen and nothing changes afterwards.	No report — the drill's actual product is missing.	Write up what went well, what didn't, and what would be handled differently. Examining the outcomes is the valuable part.
Drill scenarios are invented from scratch each time.	An unused source of realistic failures.	Build them from postmortems, use real tools, and consider breaking the test environment so the troubleshooting is real.
Only the on-call engineers have ever practised.	The circle of responders is wider than the rota.	Include everyone who could be swept in — developers, customer support, marketing partners.
The full framework is too much for the team.	Correct, and not a reason to have none.	Take the parts that matter: delegation and escalation are allowed, mitigation comes first, and the three roles are defined.
Exercises feel nothing like the real thing.	Calm rehearsal cannot manufacture urgency.	Add a time-bound simulation. The pressure and the need to talk to each other are the parts that do not transfer from a tabletop.

Reading a postmortem

WHAT YOU OBSERVE	MOST LIKELY DIAGNOSIS	WHAT TO DO
A named individual appears in the root cause.	Finger pointing.	Rewrite it as a system gap. Blame makes people risk-averse and motivates them to cover up the facts you needed next time.
“Which is ridiculous”, “careless”, an exclamation mark.	Animated language in what should be a factual artifact.	Replace judgement with verifiable data. It distracts from the message and erodes psychological safety.
The only number in the document is the duration.	Key details omitted.	Add impact figures — even a well-informed estimate. If you cannot measure it, you cannot know it is fixed.
“Improve alerting.” “Make automation better.”	Ambiguous action items with no success criteria.	Rewrite until a stranger could tell whether it is done — for example, alert when more than X% of machines disappear.
Every action item is the same priority.	No way to determine what to tackle first.	Prioritise them, and require at least one P0 or P1 for a user-affecting outage.
One item has a ticket; the rest do not.	No formal tracking.	File them all. Untracked items are forgotten, and forgotten items become outages.

Every item is mitigative.	Recurrence has not been addressed.	Add preventative items per theme. Survivable is not the same as impossible.
“Train people not to run that command.”	An attempt to fix humans.	Ask what check would have rejected the command. Changing systems and processes is more reliable than changing behaviour.
Four or five owners on the front page.	Diffused ownership.	One owner, many collaborators — a single point of contact for the document, the follow-up and the completion.
Shared with the team that owns the service.	Limited audience.	Publish company-wide by default, and consider customers. The value is proportional to the learning it creates.
Published two months after the incident.	Delayed publication.	Publish in days. Details decay, and readers fill the gap with the products of their imagination.
People are relieved not to be involved in one.	Avoiding association — the culture is failing.	Review high-visibility documents for blameful prose, and share strong examples along with how the authors were rewarded.
“Blameless is fine, but someone must have known.”	Leadership failing to reinforce the culture.	Redirect generically: were there warning signs, and why might we have dismissed them? People act in good faith on the information they have.
Quality is dropping and documents arrive late.	No time allocated to write them.	Prioritise the work and track completion. These are letters to future team members; the quality bar teaches them what to do.
The same outage recurs though every item closed.	The right items were never captured.	Ask the five questions, then step back to overall service health and bring the collaborators from each occurrence together.

Reverse mapping — you see a practice, what is it there for?

The other direction. Use this when auditing a team's incident practice rather than diagnosing a live incident: for each thing you find them doing, say what it was buying and what it does *not* buy.

WHAT YOU FIND PEOPLE DOING	WHAT IT COUNTERS	WHAT IT DOES *NOT* COVER
Declaring early and often	Uncoordinated parallel investigation, silence toward the audience, and a response nobody can hand over.	The fault itself. Declaring creates the structure; it does not shorten the diagnosis by a minute on its own.
A single named commander	Unowned work, because the commander holds every role not yet delegated.	Expertise. A commander who is also the only person who understands the system is still a bottleneck.
A communications lead	Interruptions to the people fixing it, and an organisation that assumes nothing is being done.	Anything technical. It is deliberately the one role that touches no systems.
One channel, in real time	Side conversations, contradictory decisions, and a lost timeline.	Depth. Dense debugging drowns it, which is what the shared document is for.
The shared working document	Duplicated investigation and theories lost in the scroll.	Decisions. Keep those in the channel, or nobody knows what was agreed.
Generic mitigation	The hours between knowing where the problem is and knowing what it is.	Recurrence, and precision. It is blunt by design and it prevents nothing.

Pre-built mitigation tooling	The discovery, mid-incident, that the blunt instrument would take an hour to build.	Knowing which tool you need — that comes from reading old postmortems.
Root-cause analysis	Recurrence.	The current impact. Doing it first is the single most expensive ordering mistake available.
Agreed channel, templates, contacts, criteria	The first fifteen minutes of every incident, spent deciding things twice.	Whether anyone can actually use them — only a drill establishes that.
Drills, run periodically	Gaps in the response that no document reveals, and hesitation under stress.	Nothing at all, unless a report follows. The outcomes are the product.
Rotating the commander on a long incident	Exhaustion, and an investigation circling its own eliminated theories.	Continuity. Every rotation is a handover, which is why the record has to exist.
Blameless language	Risk aversion and the covering-up of facts, which is what costs you the *next* postmortem.	Accountability — that comes from a single owner and owned, tracked action items.
Owner, priority and ticket per action item	Items quietly forgotten, and no way to decide what to do first.	Whether the item is the right one. All of them can close and the outage still recur.
Publishing company-wide within days	Stale details, an audience inventing its own account, and learning trapped in one team.	Follow-through. A widely read document with no closed actions is indistinguishable from no document.
Rewarding action-item closeout	The unvirtuous cycle where postmortems are written and never finished.	The writing itself, which still needs time explicitly allocated to it.

SECTION 8

Numbers worth remembering

NUMBER	WHAT IT IS
1968	When the Incident Command System was established, by firefighters, as a way to manage wildfires. Companies later adapted it to computer and system failures.
3 Cs	Coordinate the response effort; communicate between responders, within the organisation and to the outside world; maintain control over the response. When incident response goes wrong, the culprit is likely in one of these.
3 roles	Incident Commander, Operations Lead, Communications Lead — a hierarchy, with the latter two reporting to the commander, who holds every role not yet delegated.
4 principles	Maintain a clear line of command; designate clearly defined roles; keep a working record of debugging and mitigation as you go; declare incidents early and often.
4 steps	The order of an active incident: assess the impact, mitigate the impact, perform root-cause analysis, and afterwards fix the cause and write the postmortem.
3 or more	The number of responders at which you should start a collaborative document of working theories, eliminated causes and useful debugging information.
6 h 40 m	How long cluster creation failed across one continent in the large worked case — the failure had begun about an hour before anyone was paged. The page came at 6:41 a.m., the incident was declared at 7:06, users were notified at 7:24, and all zones were back to 0% error at 12:11 p.m.
9:56 → 10:00	The cost of hesitating. A plausible cause was identified at 9:56 a.m.; a generic rollback to a known good state at that point would have mitigated the incident by **10 a.m.** The bespoke rebuild was instead at 90% at 10:59, when a second bad path forced a restart; the fix landed at 11:59.
90%	How far the bespoke rebuild had got at 10:59 when a second bad path was discovered — forcing a restart. Bespoke work can be reset to zero by one late finding.
41 · 26,000 · 7 · 6 · 28	The scale of that one incident: 41 unique people in the channel, 26,000 words of logs, seven task forces (up to four simultaneously), on-calls summoned from six teams, and 28 action items in the postmortem.
2.5 hours	How old that incident was when the channel held a dozen participants, still with no root cause and no mitigation. A formal response structure went in two hours after the first page; the judgement afterwards was that it should have been there from the page itself.
×3	In the mishandled case, quota was raised three separate times, stopping the impact each time and preventing recurrence none of them. Only the root cause did that.
48 a day	The symptom in that case: training data was being fetched every 30 minutes instead of once per day, exceeding the service's quota. The underlying bug caused fetches roughly 50 times more often than expected.
25% → 100%	The rollout that kept going: halted at 25% when the anomaly was first noticed, restarted, past 50% mid-week, and at 100% over a weekend when developers were not readily available — at which point users lost half their requests.
4 strikes, 2 minutes	The power-outage case: lightning struck the grid near a datacentre four times within two minutes, and the third and fourth strikes were too closely spaced for the disk-tray batteries to swap over.
0.000001%	The data lost from running machines in that incident — and only from running instances — because storage was replicated across multiple datacentres.
4 audiences	What the communications lead reported to in that incident: company leaders, teams with storage concerns, external customers, and specific customers who had filed support tickets.
3 objectives	Two were clearly defined, well understood and documented, and went to the operations lead. The third was vague and covered by no procedure, so the commander assigned a dedicated coordinator and organised engineers to build new tools.

every 4 hours	How often on-call engineers and the commander were rotated during an incident that ran past ten hours — for rest, and to bring new ideas into the response team.
2 hours	How long that incident ran across three teams before anyone declared a major incident. The commander assembled the response team six minutes after the declaration.
40 minutes	The main outage in the postmortem case study, on 11 August 2014 — after which reconstruction and residual problems ran on for nearly two days.
4 months vs 4 days	The two publication dates for that same outage. The weak one appeared four months after the incident, the strong one four days after it — and the longer the wait, the more of the detail a document can no longer capture.
3 years later	When a similar incident happened. The action items from the strong postmortem had dramatically reduced the blast radius and the rate of the repeat — the only real evidence that any of this works.
5 themes	How that postmortem's action items were grouped: prevention and risk education, emergency response, monitoring and alerting, provisioning, and cleanup — which makes owners and priorities easier to assign.
P0 / P1	The stated floor: all postmortems following a user-affecting outage must have at least one P0 or P1 bug associated with them, because to users a postmortem without subsequent action is indistinguishable from no postmortem.
25% / 23% / 29%	A monitoring safety check that did not fire when 23% of targets were removed, but did fire when the same contents — 29% of what remained — were added back, delaying the restoration of monitoring by 30 minutes. An example of a check calibrated to the wrong direction.
2 × a year	How often one organisation runs fix-it weeks, where engineers who close the most postmortem action items get recognition — one of several ways to reward closeout rather than authorship.

Glossary

TERM	MEANING
Resolving an incident	Mitigating the impact and/or restoring the service to its previous condition.
Managing an incident	Coordinating the efforts of responding teams efficiently, and ensuring communication flows both between responders and to those interested in the incident's progress.
Incident Command System	The framework the practice derives from: established in 1968 by firefighters, providing standardized ways to communicate and fill clearly specified roles during an incident. It scales up or down with the size of the company and the incident.
The three Cs	Coordinate, communicate, control — the common goals of every incident response framework, and where the culprit is likely to lie when a response goes wrong.
Incident Commander	Leads the response and directs its high-level state; commands and coordinates, delegates, communicates, stays in control, and works with responders to resolve. Assumes all roles not yet delegated.
Operations Lead	Responds to the incident by applying operational tools to mitigate or resolve it. May lead a team, which expands or contracts as needed.
Communications Lead	The public face of the response: periodic updates to the response team and stakeholders, and managing inquiries. Can be subsumed back into the commander if the incident becomes small enough.
Declaring an incident	The act that creates the structure. The person who declares it typically steps into the commander role. The principle is to declare early and often.
War room	The single place — physical or virtual — where all responders gather and communicate in real time. Centralized communication is a stated principle of the protocol.
Working document	A collaborative document started when three or more people work on an incident, listing working theories, eliminated causes and useful debugging information, so it isn't lost in the conversation.
Generic mitigation	An action first responders take to alleviate pain before the root cause is fully understood — a rollback, or reconfiguring load balancers away from a region. Blunt instruments that may cause other disruptions.

Bespoke mitigation	A fix built for the specific fault, which cannot begin until the fault is understood and can be reset by a late discovery.
Mitigation tooling	General-purpose tools that make a generic mitigation a single action. They take engineering time, so the right time to create them is before an incident, not during one.
Escalation	Bringing in more people or other teams, by documented path or by broadcasting to prepared lists. Not a sin — especially if it helps lower customer pain.
Heroics	Individuals rescuing an incident off-rota because they happened to be reachable. Valued, but successful incident management shouldn't rely on it.
Criteria for an incident	An established list for determining whether a paging issue is indeed an incident, derived from past outages and known high-risk areas.
Drill	Practising incident management on something that is not a real emergency: a controlled company-wide exercise, a role-play of a previous postmortem, or deliberately treating a minor problem as a major one.
Scribe	The person capturing actions, owners and timestamps into the incident channel, which functions as an information ledger for the response.
Postmortem	The written record of an incident: what happened, how it was mitigated, how users were impacted, why it happened, and what will change. Its value is proportional to the learning it creates.
Blameless	Focusing on the gaps in system design that permitted the failure mode — on what went wrong rather than who — with action items aimed at improving the system instead of improving people.
Psychological safety	What animated language and public blame erode, and what makes people willing to record facts that are critical to understanding and preventing recurrence.
Action item	A change arising from a postmortem. A good one has a single owner, a priority, a tracking bug and a verifiable end state, and at least some per theme are preventative rather than mitigative.
Root cause and trigger	The underlying defect and the specific event that set it off — identifying both is one of the most important reasons to write a postmortem, and it should explore the lower-level details rather than summarise them.
Idempotent	Safe to run more than once with the same result. The worked outage happened because a decommission workflow was not, and the follow-up work made it so.
Unvirtuous cycle	What you risk by rewarding engineers for writing postmortems but not for closing the associated action items: documents that are written and never finished.
Letters to future team members	How postmortems are described when arguing for a consistent quality bar — a subpar one teaches future teammates a bad lesson.

Answer key

PART 1 · Q1 — THE PAYMENTS API THAT WAS NEVER DECLARED

(a) **Resolving** the incident — mitigating the impact and/or restoring the service to its previous condition — and **managing** it: coordinating the efforts of the responding teams efficiently, and ensuring communication flows both between the responders and to those interested in the incident's progress. Five people are resolving. Nobody is managing, and that is the job with no owner. (b) The four principles are: maintain a clear line of command; designate clearly defined roles; keep a working record of debugging and mitigation as you go; and declare incidents early and often. All four are broken. There is no line of command, because nobody is leading and four people joined without being asked. There are no roles — in particular nobody is doing communications, which is why the support team is interrupting the one person best placed to fix it. The record is a ticket used as a notebook rather than a working document of theories and eliminated causes, which is how two engineers came to rule out the same subsystem independently; with five responders that document should exist. And no incident has been declared. (c) The reasoning treats declaring as an escalation whose cost must be justified by severity. It is the opposite: declaring is the act that *supplies* the tools — the command line, the roles, the shared channel and the record — so waiting does not defer a cost, it forfeits three specific benefits. Declaring early prevents miscommunication between the separate groups working the problem; makes root-cause identification and resolution occur sooner; and loops relevant teams in earlier, which makes external communication faster and smoother. Each of those is already being lost here. The principle is early **and often**, precisely because standing a response down is trivial and starting one ninety minutes late is not. (d) Something close to: “I'm declaring an incident, and I'm the Incident Commander.” “You're Operations Lead — you own the merchant errors, and everything you rule out goes in the shared doc.” “You're Communications Lead — support and the status page come to you, not to us; everyone else into the incident channel now.”

PART 1 · Q2 — NINETEEN PEOPLE AND ONE EXHAUSTED COMMANDER

(a) By default **the commander assumes all roles that have not been delegated yet**. Nothing has been delegated, so this person is simultaneously Incident Commander, Operations Lead and Communications Lead. The consequence is that the one job only they can do — coordinating — is the one being starved, because it has no deadline attached to it while the director and the status page do. The visible symptom is the duplicated forty minutes: coordination is what prevents two people doing the same elimination, and coordination is not happening. (b) Two moves are available, and both are normal. The commander may **hand off the commander role** to the experienced colleague and take the Operations Lead role themselves — cost: a handover, at a bad moment, with all the context to transfer; benefit: the response is run by whoever is best at running it, which need not be whoever noticed first. Or they may **assign the Operations Lead role** to someone else and stay in command — cost: they keep the coordination load and, unless they also delegate communications, the director too. The large worked case took the first option, transferring command before the scheduled shift handover specifically because the incoming person had more incident-response experience. (c) The **working document**, started once three or more people are working an incident. It lists working theories, eliminated causes, and useful debugging information such as error logs and suspect graphs, and its stated purpose is to preserve that information so it does not get lost in the conversation. With nineteen participants it is long overdue. (d) Move the **detailed debugging into the shared document** and keep the **channel as the hub for decision making**. That is exactly the split the large worked case adopted once discussion became too dense for the channel, and it works because the two kinds of traffic have different readers: detail is read by whoever picks up the thread, decisions must be read by everyone.

PART 2 · Q1 — THREE SITES, ONE CONTINENT, A RECENT PUSH

(a) The order is: **1.** assess the impact; **2.** mitigate the impact; **3.** perform a root-cause analysis; **4.** after the incident is over, fix what caused it and write a postmortem. Step 1 is done — a fifth of queries failing, localised to three sites. The team is working on step 3, and has skipped step 2 entirely. That inversion is the failure the whole ordering exists to prevent: first responders must prioritise mitigation above all else, or time to resolution suffers. (b) The rule is that **to mitigate an incident you don't have to fully understand the details — you only need to know the location of the root cause**. Enough is known. The failures are confined to three named sites in one continent, and a configuration push went to exactly those sites shortly before the errors began. That is a location plus a correlation with a recent change, which is precisely the situation the two canonical generic mitigations are for: roll back a recent release when an outage is correlated with the release cycle, or reconfigure load balancers to avoid a region when errors are localised. Both are available here. (c) The objection correctly identifies something real: generic mitigations are **blunt instruments and may cause other disruptions to the service**, and rolling back the configuration genuinely does undo a week of unrelated changes. It does not change the decision because that bluntness is the price of not needing a diagnosis, not an argument against paying it — the alternative on offer is another ninety minutes of a fifth of queries failing. Customers do not care whether you understand the outage; what they want is to stop receiving errors. The unrelated changes can be reapplied on a calm afternoon. (d) The targeted patch should continue **in parallel**, not instead. The error is running bespoke work as the only thing standing between users and the impact, and the worked case shows why: the equivalent effort there was a rebuild taking about an hour, which reached 90% completion before the team discovered another location using the bad path and had to

restart the build. Bespoke work can be reset to zero by a single late discovery, and it cannot begin at all until the fault is understood. Note also that the rollback ends the impact but not the incident: mitigation is not prevention, so the investigation continues to root cause.

PART 2 · Q2 — THE QUEUE THAT OVERFLOWS EVERY FORTNIGHT

(a) The three previous fixes **mitigated the impact** — the backlog drained and the alert cleared — and did nothing about **recurrence**, which requires the root cause. This is the mishandled worked case exactly: mitigation successfully stopped the impact on three separate occasions, each time by raising a quota, but the team could only prevent the issue from happening again once they discovered the root cause. Stopping the impact and stopping the repeat are different achievements, and only the second needs a diagnosis. (b) Two decisions repeat that case's mistakes. First, **continuing the rollout after mitigating**: after the first mitigation it would have been better to postpone the rollout until the root cause was fully determined, which is what avoids the eventual major disruption. The corrective is to hold the rollout until the cause is understood. Second, **rolling out when the people who understand it are not available** — the worked case's rollout reached 100% on a weekend, against the practice of performing rollouts only during business days so developers are around if something goes wrong. The corrective is to conduct rollouts during business hours. (c) Because **successful incident management shouldn't rely on heroic efforts of individuals** — the question that makes it a finding is *what if they had been unreachable?* The organisation was one unanswered message away from having no response at all, and the arrangement also taps someone during their own time, against a good work-life balance. The two structural alternatives are stated: conduct the rollouts during business hours, or organise an on-call rotation that provides **paid** coverage outside business hours. (d) Stop the rollout where it is. It should not resume until the root cause of the backlog is fully determined — and, given that this is the fourth occurrence, until whatever the root cause turns out to be has an owned fix rather than another limit increase. When it does resume, it should resume on a business day with the people who understand the system available.

PART 3 · Q1 — AN EXCELLENT POLICY NOBODY HAS RUN

(a) Four things should have been settled in advance, and each maps to one delay. The **communication channel** — agreed and practised beforehand — removes the eleven minutes spent choosing between tools. Two or three **ready-to-use templates**, with the on-call knowing how to send them and the review and approval route agreed, remove the forty minutes of drafting and rewriting. A **prepared list of contacts** to email or page removes the search for the payments escalation. And **established criteria for what counts as an incident** remove the quarter-hour argument. (b) For the channel: **no Incident Commander wants to make this decision during an incident** — and the further instruction is to practise using it so there are no surprises, and to prefer one the team is already familiar with, because familiarity is what removes hesitation. For the templates: **no one wants to write these announcements under extreme stress with no guidelines.** Neither reason is about quality of judgement; both are about the cost of deciding at all at the worst moment. (c) “Re-read the incident policy” mistakes documentation for preparation. A document tells people what they should do; practice determines what they will do. Staying calm and following a response structure during an emergency **takes practice, and practice builds muscle memory**, which is what supplies confidence in a real emergency. Re-reading rehearses nothing and closes none of the four gaps. The correct action items are the four concrete decisions above, plus a drill and — crucially — the report that follows it. (d) The criteria argument is the most consequential because it delays the **declaration itself**, and everything else in the response exists only after that: the command line, the roles, the shared channel, the record. A team that cannot agree whether an event qualifies cannot declare early, so the whole structure arrives late. The criteria should be derived by **looking at past outages and taking known high-risk areas into consideration** — which this organisation, having had the outage described, is now in a position to do.

PART 3 · Q2 — A TABLETOP WITH NO REPORT, AND A NINE-HOUR INCIDENT

(a) Three scales are available: a **company-wide resilience test**, which creates a controlled emergency that does not actually impact customers and which teams respond to as if it were real; a **role-playing exercise** in which engineers reenact a previous incident in its documented roles; and **intentionally treating minor problems as major ones** requiring a large-scale response, which lets the team practise with the real procedures and tools in a real-world situation at lower stakes. The third costs nothing and can be run this week, on the next minor problem that arrives; the role-play is nearly as cheap if any postmortems exist. (b) What is missing is the **report afterwards**, detailing what went well, what didn't, and how things could have been handled better. Its product is the list of **gaps in incident management**: the most valuable part of running a drill is examining the outcomes, because once you know what the gaps are you can work toward closing them. Without it the exercise is a conversation that leaves no trace, which is also why drills are far more useful when run periodically rather than once. (c) Build the scenario from a **postmortem** — postmortems contain plenty of ideas for incident management drills. They describe failures the system has actually produced rather than ones somebody imagined, they come with a known account of what happened for comparison, and they reuse work already done. Two related instructions: use real tools as much as possible, and consider breaking your test environment so the team performs real troubleshooting. (d) The skipped practice is **rotating responders, including the commander** — one organisation rotated both every four hours through an incident that ran past ten hours. Its two stated benefits are different in kind: it **encouraged engineers to get rest**, and it **brought new ideas into the response team**. The second is what would have prevented hour seven's re-testing of an eliminated hypothesis, since the person who eliminated it seven hours ago is the least

likely person to notice they are doing it again. (The shared working document guards the same failure from the other direction, by recording what has already been ruled out.)

PART 4 · Q1 — THE BILLING POSTMORTEM

(a) Seven distinct failure modes. **Counterproductive finger pointing**: a named individual identified as not having escalated, and the platform team blamed for the missing documentation. **Animated language**: “frankly astonishing” and the exclamation in “Where we got lucky” — a postmortem is a factual artifact that should be free from personal judgements and subjective language. **Weak action items**, with four separate defects: ambiguous phrasing (“Make failover more reliable”, “Improve on-call training”) that is open to interpretation and unmeasurable; all items at equal priority, so there is no way to determine which to tackle first; missing owners on two of three; and only one tracking bug. **An action item aimed at humans** rather than systems. **Missing ownership**: five owners on the document. **Limited audience**: shared with the platform team alone. There is also no impact data anywhere in the section shown, which is **key details omitted**. (b) Highlighting individuals **leads team members to become risk-averse because they're afraid of being publicly shamed**, and — the part that matters operationally — **they may be motivated to cover up facts critical to understanding and preventing recurrence**. That makes it a practical problem rather than an etiquette one: the cost is not hurt feelings but missing information in the *next* postmortem, which is the document you will need to prevent the next outage. Naming Priya here buys nothing and quietly degrades every future write-up in the organisation. (c) Rewritten, each with a single owner, a priority, a tracking bug and a verifiable end state, and aimed at the system: “**Add an automated failover exercise to the weekly release pipeline that fails the build if failover exceeds N minutes**” — P1, one owner, ticket. “**Page the secondary on-call automatically when an incident is unmitigated after 30 minutes**” — P1, one owner, ticket. “**Write the failover runbook and validate it by executing it end to end in a drill**” — P1, one owner, ticket. The item to delete outright is “improve on-call training so people escalate sooner”: **trying to change human behavior is less reliable than changing automated systems and processes**, so it is replaced by the automatic escalation above rather than kept. Note also that this was a user-affecting outage, so at least one item should be P0 or P1. And the runbook item's owner should be whoever will do the work, not the person who was on call. (d) The five owners should be **one owner and multiple collaborators** — a single point of contact responsible for the postmortem, the follow-up and the completion, because declaring official ownership results in accountability, which leads to action. The platform-team-only audience should be **everyone at the company by default**, shared as widely as possible and perhaps even with customers, because the value of a postmortem is proportional to the learning it creates.

PART 4 · Q2 — THREE SIGNALS A DIRECTOR NOTICED

(a) Two harms. A prompt postmortem **tends to be more accurate because information is fresh in the contributors' minds**, so a seven-to-ten-week delay loses detail — and had the worked case's outage recurred while its weak document was still pending, which the outage did in reality do, the team would likely have forgotten details a timely one would have captured. And the people affected are waiting for an explanation and a demonstration that you have things under control: **the longer you wait, the more they will fill the gap with the products of their imagination**, which seldom works in your favour. The underlying cause named is **lacking time to write postmortems**: quality postmortems take time, and when a team is overloaded the quality suffers, producing subpar documents with incomplete action items that make recurrence far more likely. On quality bars: **postmortems are letters you write to future team members**, so it is very important to keep a consistent bar, lest you accidentally teach future teammates a bad lesson. The remedy is to prioritise postmortem work, track completion and review, and allow teams adequate time to implement the action plan. (b) Something close to: “I'm sure everyone had the best intent, so to keep it blameless, maybe we ask generically whether there were any warning signs we could have heeded, and why we might have dismissed them.” It moves the narrative in a more constructive direction without contradicting the vice-president. The justifying principle is that **individuals act in good faith and make decisions based on the best information available**, so **investigating the source of misleading information is much more beneficial to the organisation than assigning blame** — the interesting question is why the warning did not reach the people deciding, which is a system question. (c) Repeating incidents mean it is time to dig deeper, and the questions are: are action items taking too long to close? Is feature velocity trumping reliability fixes? Are the right action items being captured in the first place? Is the faulty service overdue for a refactor? Are people putting sticking plasters on a more serious problem? When every item is closing on time, the first question is answered and the remaining ones become the live ones — most pointedly whether the right items were ever captured, and whether the service needs a refactor rather than more patches. Having uncovered a systemic process or technical problem, step back and consider overall service health, and bring the postmortem collaborators from each similar incident together to decide the course of action. (d) Two that target closeout and learning rather than authorship. **Reward action item closeout explicitly**, keeping incentives balanced between writing the postmortem and successfully implementing its action plan — this prevents the unvirtuous cycle of unclosed postmortems you get from rewarding the writing alone. And **gamification**: a scoreboard or burndown of open action items, of the kind used in twice-yearly fix-it weeks where those who close the most receive recognition — this prevents items ageing invisibly by making the backlog itself public. Equally acceptable: rewarding the resulting organisational change with peer bonuses, positive reviews or promotion, which prevents learning staying inside one team; and holding postmortem owners up as experts, which counteracts people avoiding association with a write-up.

SECTION 10

Spaced-review tracker

Review at **day 1, 3, 7, 16, 35**. Write the date in the box when you pass. On a fail, reset that row to a 1-day interval and start again.

ITEM	DAY 1	DAY 3	DAY 7	DAY 16	DAY 35	FAILS
1 · Resolving vs managing an incident						
2 · The four basic principles						
3 · Where the framework came from, and its scaling						
4 · The three Cs, and what they are goals for						
5 · The commander's default over undelegated roles						
6 · Operations and communications leads						
7 · Which role collapses back, and when						
8 · The three things declaring early buys you						
9 · Centralized communication, and the channel/doc split						
10 · The working document: when, and what it holds						
11 · The four steps of an active incident						
12 · Location, not details — and why it saves hours						
13 · Generic mitigation: two examples and the price						
14 · Bespoke mitigation, and how it gets reset						
15 · When to build mitigation tooling, and how to know which						
16 · Mitigation is not prevention — the three-quota case						
17 · Heroics, and the two structural alternatives						
18 · Delegating the documented, coordinating the vague						
19 · What response training actually teaches						
20 · The four things to settle in advance						
21 · The two symmetrical assumptions of silence						
22 · Where incident criteria come from						
23 · The three scales of drill						
24 · The actual product of a drill						
25 · Rotation: both reasons, and which is a technique						
26 · The worked outage: empty list, no filter						
27 · The eight ways a postmortem fails						
28 · Four defects of a weak action-item list						
29 · Why blame is a practical problem						
30 · Blameless but owned — reconciling the two						
31 · The seven qualities of a good postmortem						
32 · Audience and promptness, and the reasons for each						
33 · The four signs the culture is failing						
34 · Master matrix — commanded × changed						

Final quiz — no notes, twenty minutes, write the answers

1. Define resolving an incident and managing an incident, give the four basic principles of incident response, and say what a pre-agreed structure is actually protecting.

2. Give the three Cs, say what they are goals for and what they are not, and state where the culprit usually lies when a response goes wrong.

3. Name the three main roles, say how they are organised, give the default rule about roles that have not been delegated, and say which role can be absorbed back and under what condition.

4. Give the three specific benefits of declaring an incident early, and explain why declaring should be understood as creating the response rather than escalating it.

5. Give the four steps of an active incident in order, say which is top priority and what happens if it is not, and state the rule about details versus location.

6. Define a generic mitigation, give two examples, state the price it carries, and say when the tooling for one should be built and how you find out which tooling you need.

7. Explain the difference between mitigating an incident and preventing its recurrence, using the case where the impact was stopped three times, and say what should have happened to the rollout.

8. Name the four things to settle before an incident, give the stated reason for two of them, and give the two symmetrical assumptions people make when you tell them nothing.

9. Give the three scales of incident drill, say what makes a drill worth running and what makes one worthless, and give the two reasons for rotating responders during a long incident.

10. Give the four characteristics of a good action item, state the rule about changing humans versus changing systems, and explain the mechanism by which blame in a postmortem damages the next one.